# PR 3: `@x402/moi` mechanism package

*Review document, 10 September 2026. Prepared for internal review before filing.*

## Page 1: what this PR is

**Target repository:** `x402-foundation/x402` (public). New package `typescript/packages/mechanisms/moi/`, plus the e2e catalog, a changeset, a publish workflow, and small registrations in three existing e2e files and three existing example files.

**PR title:** feat(moi): add `@x402/moi` mechanism package for the `exact` scheme

**Why it exists:** it is the code that implements PR 2. A seller installs it and registers `moi:*`; a buyer registers the client half; the facilitator half confirms transfers by reading them back from a node. The layout mirrors `@x402/stellar`, the closest merged non-EVM mechanism.

### High-level requirements

From the repo's contributing guide and the Stellar package it is modelled on.

| Requirement | Where it is met |
|---|---|
| Three halves (client, server, facilitator) as separate entry points, each implementing core's `SchemeNetwork*` interface | `src/exact/client`, `src/exact/server`, `src/exact/facilitator` |
| Every MUST rule of the spec enforced, with namespaced `errorReasons` | `src/exact/facilitator/scheme.ts`; table in `README.md` |
| Unit tests with 70% coverage; integration tests against a live network | `test/unit/` (36 tests), `test/integrations/` |
| Package README with install and usage for each half | `README.md` |
| e2e catalog entry plus registration in the e2e server, client and facilitator | `e2e/config/mechanisms_moi.json`; edits in section 3 |
| `examples/typescript/*/advanced/all_networks.*` entries, alphabetical | edits in section 4 |
| Changeset under `typescript/.changeset/` | `moi-exact-mechanism.md` |
| Publish workflow | `.github/workflows/publish_npm_scoped_x402_moi.yml` |
| Lint and format with upstream's config; `tsc` clean | `eslint.config.js`, `.prettierrc` |
| Commits GPG-signed, conventional prefixes, | PR checklist |

| Requirement | Where it is met |
|---|---|
| AI assistance disclosed, payment logic personally reviewed | |

## Status

- Verified 10 September against the real dependencies from npm (`@x402/core@2.25.0`, `js-moi-sdk@0.9.0-rc2`, `js-polo@0.1.5`): `tsc`, ESLint and Prettier clean; 36 of 36 unit tests pass; the live read-back tests pass against Voyage devnet.
- **Before filing:** run the pay-prove-settle round trip against moipod v0.13.0 with a funded devnet wallet (`pnpm test:integration`). It is written and skips without credentials.
- **Sequencing:** files after PR 2 merges, which files after PR 1 merges. If PR 1 changes the reference format, `src/constants.ts` changes with it.
- Every code change goes through Gokul's review; the package leaves the org only after Rahul's.

## Index


1. PR description (as it will be posted)
2. New files, in package order, each with a context note
    - 2.1 `typescript/packages/mechanisms/moi/package.json`
    - 2.2 `typescript/packages/mechanisms/moi/tsconfig.json`
    - 2.3 `typescript/packages/mechanisms/moi/tsup.config.ts`
    - 2.4 `typescript/packages/mechanisms/moi/vitest.config.ts`
    - 2.5 `typescript/packages/mechanisms/moi/vitest.integration.config.ts`
    - 2.6 `typescript/packages/mechanisms/moi/eslint.config.js`
    - 2.7 `typescript/packages/mechanisms/moi/.prettierrc`
    - 2.8 `typescript/packages/mechanisms/moi/.prettierignore`
    - 2.9 `typescript/packages/mechanisms/moi/README.md`
    - 2.10 `typescript/packages/mechanisms/moi/CHANGELOG.md`
    - 2.11 `typescript/packages/mechanisms/moi/src/index.ts`
    - 2.12 `typescript/packages/mechanisms/moi/src/constants.ts`
    - 2.13 `typescript/packages/mechanisms/moi/src/types.ts`
    - 2.14 `typescript/packages/mechanisms/moi/src/utils.ts`
    - 2.15 `typescript/packages/mechanisms/moi/src/shared.ts`
    - 2.16 `typescript/packages/mechanisms/moi/src/signer.ts`
    - 2.17 `typescript/packages/mechanisms/moi/src/reader.ts`
    - 2.18 `typescript/packages/mechanisms/moi/src/defaultAssets.ts`
    - 2.19 `typescript/packages/mechanisms/moi/src/exact/index.ts`

- 2.20 typescript/packages/mechanisms/moi/src/exact/client/index.ts
- 2.21 typescript/packages/mechanisms/moi/src/exact/client/scheme.ts
- 2.22 typescript/packages/mechanisms/moi/src/exact/server/index.ts
- 2.23 typescript/packages/mechanisms/moi/src/exact/server/scheme.ts
- 2.24 typescript/packages/mechanisms/moi/src/exact/facilitator/index.ts
- 2.25 typescript/packages/mechanisms/moi/src/exact/facilitator/scheme.ts
- 2.26 typescript/packages/mechanisms/moi/test/unit/facilitator.test.ts
- 2.27 typescript/packages/mechanisms/moi/test/unit/reader.test.ts
- 2.28 typescript/packages/mechanisms/moi/test/unit/server.test.ts
- 2.29 typescript/packages/mechanisms/moi/test/integrations/exact-moi.test.ts
- 2.30 typescript/.changeset/moi-exact-mechanism.md
- 2.31 e2e/config/mechanisms_moi.json
- 2.32 .github/workflows/publish_npm_scoped_x402_moi.yml

3. Edits to existing e2e files (E2E-REGISTRATION.md)
4. Edits to existing example files (EXAMPLES-ENTRIES.md)
5. Verification record (STATUS.md)

# 1. PR description

## PR description — x402-foundation/x402 (TypeScript mechanism)

**Title:** feat(moi): add `@x402/moi` mechanism package for the `exact` scheme

Implements `specs/schemes/exact/scheme_exact_moi.md` as `typescript/packages/mechanisms/moi`, mirroring the layout of `@x402/stellar`.

MOI uses the `upfront` payment flow: the buyer settles a protocol-native asset transfer from its own account and proves it with a signed claim bound to one resource and one network. Core does not call `/verify` under this flow, so every rule in the spec is enforced in `settle`, which confirms the transfer by reading `moi.InteractionByHash` and `moi.InteractionReceipt` and POLO-decoding the operation's calldata. The spec explains why `upfront` was chosen over the `authorization` and `escrow` flows MOI also supports.

The facilitator holds no keys and signs nothing, so `getSigners()` returns an empty list and in-process self-facilitation is the intended deployment. A JSON-RPC chain reader (`createMoiChainReader`) is included so a facilitator runs without further glue: it verifies claim signatures through js-moi-sdk, derives participant identifiers the way the SDK wallet does, and reads settled transfers back.

Tests: 36 unit tests, including a POLO decode of a real devnet transfer's calldata and signature round trips with the SDK's own ECDSA. The integration file reads a settled Voyage devnet transfer back with network access alone, and runs a full pay-prove-settle round trip when a funded buyer is configured.

Also included: `e2e/config/mechanisms_moi.json`, the scheme registrations in the e2e server, client and facilitator modules (`e2e/servers/typescript/config.ts`, `e2e/clients/typescript/client.ts`, `e2e/facilitators/typescript/index.ts`), a changeset under `typescript/.changeset/`, and a publish workflow.

- ☐ Scheme spec merged first (PR 2)
- ☐ CAIP-2 namespace merged with CASA, and the `moi:*` references confirmed
- ☐ Round trip run against moipod v0.13.0 with a funded devnet wallet
- ☐ Commits GPG-signed, conventional prefixes
- ☐ AI assistance disclosed: this package was drafted with AI assistance and human-reviewed
- ☐ Payment and signature logic personally reviewed
- ☐ `examples/typescript/*/advanced/all_networks.*` entries added, alphabetically before `near`
- ☐ Changeset added
- ☐ Internal review (Rahul) complete

## 2. New files

### 2.1 `typescript/packages/mechanisms/moi/package.json`

> **Context.** Package manifest mirroring `@x402/stellar`: same scripts, same peer dependency on `@x402/core`, `js-moi-sdk ^0.9.0-rc2` and `js-polo ^0.1.5` as runtime dependencies, and three entry points (root, `exact/client`, `exact/server`, `exact/facilitator`).

```
{
  "name": "@x402/moi",
  "version": "2.25.0",
  "main": "./dist/cjs/index.js",
  "module": "./dist/esm/index.js",
  "types": "./dist/cjs/index.d.ts",
  "scripts": {
    "build": "tsup",
    "test": "vitest run --coverage",
    "test:integration": "vitest run --config
vitest.integration.config.ts",
    "test:all": "vitest run --config vitest.config.ts && vitest run --
config vitest.integration.config.ts",
    "test:watch": "vitest",
    "watch": "tsc --watch",
    "format": "prettier -c .prettierrc --write \"**/*.
{ts,js,cjs,json,md}\"",
    "format:check": "prettier -c .prettierrc --check \"**/*.
{ts,js,cjs,json,md}\"",
    "lint": "eslint . --ext .ts --fix",
    "lint:check": "eslint . --ext .ts"
  },
  "keywords": [
    "x402",
    "payment",
    "protocol",
    "moi",
    "mas0"
  ],
  "license": "Apache-2.0",
  "author": "x402 Foundation",
  "repository": "https://github.com/x402-foundation/x402",
  "description": "x402 Payment Protocol MOI Implementation",
  "engines": {
    "node": ">=22.0.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.24.0",
    "@types/node": "^22.13.4",
    "@typescript-eslint/eslint-plugin": "^8.29.1",
    "@typescript-eslint/parser": "^8.29.1",
```

```
  "@vitest/coverage-v8": "^3.2.7",
  "eslint": "^9.24.0",
  "eslint-plugin-import": "^2.31.0",
  "eslint-plugin-jsdoc": "^50.6.9",
  "eslint-plugin-prettier": "^5.2.6",
  "eslint-plugin-sonarjs": "^3.0.7",
  "prettier": "3.5.2",
  "tsup": "^8.4.0",
  "tsx": "^4.21.0",
  "typescript": "^5.7.3",
  "vite": "^6.2.6",
  "vite-tsconfig-paths": "^5.1.4",
  "vitest": "^3.2.7"
},
"dependencies": {
  "@x402/core": "workspace:~",
  "js-moi-sdk": "^0.9.0-rc2",
  "js-polo": "^0.1.5"
},
"exports": {
  ".": {
    "import": {
      "types": "./dist/esm/index.d.mts",
      "default": "./dist/esm/index.mjs"
    },
    "require": {
      "types": "./dist/cjs/index.d.ts",
      "default": "./dist/cjs/index.js"
    }
  },
  "./exact/client": {
    "import": {
      "types": "./dist/esm/exact/client/index.d.mts",
      "default": "./dist/esm/exact/client/index.mjs"
    },
    "require": {
      "types": "./dist/cjs/exact/client/index.d.ts",
      "default": "./dist/cjs/exact/client/index.js"
    }
  },
  "./exact/server": {
    "import": {
      "types": "./dist/esm/exact/server/index.d.mts",
      "default": "./dist/esm/exact/server/index.mjs"
    },
    "require": {
      "types": "./dist/cjs/exact/server/index.d.ts",
      "default": "./dist/cjs/exact/server/index.js"
    }
  },
```

```
    "./exact/facilitator": {
      "import": {
        "types": "./dist/esm/exact/facilitator/index.d.mts",
        "default": "./dist/esm/exact/facilitator/index.mjs"
      },
      "require": {
        "types": "./dist/cjs/exact/facilitator/index.d.ts",
        "default": "./dist/cjs/exact/facilitator/index.js"
      }
    }
  },
  "files": [
    "dist"
  ]
}
```

### 2.2 `typescript/packages/mechanisms/moi/tsconfig.json`

**Context.** Extends upstream's `tsconfig.base.json`; nothing MOI-specific.

```
{
  "extends": "../../../tsconfig.base.json",
  "compilerOptions": { "lib": ["ES2020"], "allowJs": false, "checkJs":
false },
  "include": ["src", "test"]
}
```

### 2.3 `typescript/packages/mechanisms/moi/tsup.config.ts`

**Context.** Builds ESM and CJS for each entry point, as Stellar does.

```
import { defineConfig } from "tsup";

const entry = {
  index: "src/index.ts",
  "exact/client/index": "src/exact/client/index.ts",
  "exact/server/index": "src/exact/server/index.ts",
  "exact/facilitator/index": "src/exact/facilitator/index.ts",
};

export default defineConfig([
  {
    entry,
    format: ["esm"],
    outDir: "dist/esm",
    dts: { resolve: true },
    sourcemap: true,
    target: "es2020",
    clean: true,
  },
  {
    entry,
```

```
    format: ["cjs"],
    outDir: "dist/cjs",
    dts: { resolve: true },
    sourcemap: true,
    target: "es2020",
    clean: false,
  },
]);
```

### 2.4 `typescript/packages/mechanisms/moi/vitest.config.ts`

**Context.** Unit test config with the 70% coverage thresholds upstream expects.

```
import tsconfigPaths from "vite-tsconfig-paths";
import { defineConfig } from "vitest/config";

export default defineConfig({
  plugins: [tsconfigPaths()],
  test: {
    exclude: ["test/integrations/**", "node_modules/**", "dist/**"],
    coverage: {
      provider: "v8",
      include: ["src/**/*.ts"],
      exclude: ["**/*.test.ts", "dist/**"],
      thresholds: { lines: 70, functions: 70, branches: 70,
statements: 70 },
    },
  },
});
```

### 2.5 `typescript/packages/mechanisms/moi/vitest.integration.config.ts`

**Context.** Runs only `test/integrations/`, which needs network access.

```
import { defineConfig } from "vitest/config";

export default defineConfig({
  test: {
    include: ["test/integrations/**/*.test.ts"],
    testTimeout: 20000,
    hookTimeout: 20000,
  },
});
```

### 2.6 `typescript/packages/mechanisms/moi/eslint.config.js`

**Context.** Copied verbatim from `@x402/stellar` so the package is linted the way CI lints it. Clean.

```
import js from "@eslint/js";
import ts from "@typescript-eslint/eslint-plugin";
import tsParser from "@typescript-eslint/parser";
```

```
import prettier from "eslint-plugin-prettier";
import jsdoc from "eslint-plugin-jsdoc";
import importPlugin from "eslint-plugin-import";
import sonarjs from "eslint-plugin-sonarjs";

export default [
  {
    ignores: ["dist/**", "node_modules/**"],
  },
  {
    files: ["**/*.ts", "**/*.tsx"],
    ignores: ["**/*.test.ts", "test/**/*"],
    languageOptions: {
      parser: tsParser,
      sourceType: "module",
      ecmaVersion: 2020,
      globals: {
        process: "readonly",
        __dirname: "readonly",
        module: "readonly",
        require: "readonly",
        Buffer: "readonly",
        exports: "readonly",
        setTimeout: "readonly",
        clearTimeout: "readonly",
        setInterval: "readonly",
        clearInterval: "readonly",
      },
    },
    plugins: {
      "@typescript-eslint": ts,
      prettier: prettier,
      jsdoc: jsdoc,
      import: importPlugin,
      sonarjs,
    },
    rules: {
      ...ts.configs.recommended.rules,
      complexity: ["error", 50],
      "max-lines": ["error", { max: 2000 }],
      "sonarjs/cognitive-complexity": ["error", 50],
      "import/first": "error",
      "import/order": [
        "error",
        {
          groups: [
            "builtin",
            "external",
            "internal",
            ["parent", "sibling"],
```

```
            "index",
            "object",
            "type",
          ],
          "newlines-between": "never",
          alphabetize: {
            order: "asc",
            caseInsensitive: true,
          },
        },
      ],
      "prettier/prettier": "error",
      "@typescript-eslint/member-ordering": "error",
      "@typescript-eslint/no-unused-vars": ["error",
{ argsIgnorePattern: "^_$" }],
      "jsdoc/tag-lines": ["error", "any", { startLines: 1 }],
      "jsdoc/check-alignment": "error",
      "jsdoc/no-undefined-types": "off",
      "jsdoc/check-param-names": "error",
      "jsdoc/check-tag-names": "error",
      "jsdoc/check-types": "error",
      "jsdoc/implements-on-classes": "error",
      "jsdoc/require-description": "error",
      "jsdoc/require-jsdoc": [
        "error",
        {
          require: {
            FunctionDeclaration: true,
            MethodDefinition: true,
            ClassDeclaration: true,
            ArrowFunctionExpression: false,
            FunctionExpression: false,
          },
        },
      ],
      "jsdoc/require-param": "error",
      "jsdoc/require-param-description": "error",
      "jsdoc/require-param-type": "off",
      "jsdoc/require-returns": "error",
      "jsdoc/require-returns-description": "error",
      "jsdoc/require-returns-type": "off",
      "jsdoc/require-hyphen-before-param-description": ["error",
"always"],
    },
  },
  {
    files: ["**/*.test.ts", "test/**/*"],
    languageOptions: {
      parser: tsParser,
      sourceType: "module",
```

```
      ecmaVersion: 2020,
    },
    plugins: {
      "@typescript-eslint": ts,
      prettier: prettier,
      import: importPlugin,
    },
    rules: {
      "import/first": "error",
      "import/order": [
        "error",
        {
          groups: [
            "builtin",
            "external",
            "internal",
            ["parent", "sibling"],
            "index",
            "object",
            "type",
          ],
          "newlines-between": "never",
          alphabetize: {
            order: "asc",
            caseInsensitive: true,
          },
        },
      ],
      "prettier/prettier": "error",
      "@typescript-eslint/no-unused-vars": ["error",
{ argsIgnorePattern: "^_" }],
      "@typescript-eslint/no-explicit-any": "off",
      "@typescript-eslint/member-ordering": "off",
    },
  },
];
```

### 2.7 `typescript/packages/mechanisms/moi/.prettierrc`

**Context.** Upstream formatting rules. Clean.

```
{
  "tabWidth": 2,
  "useTabs": false,
  "semi": true,
  "singleQuote": false,
  "trailingComma": "all",
  "bracketSpacing": true,
  "arrowParens": "avoid",
  "printWidth": 100,
```

```
  "proseWrap": "never"
}
```

### 2.8 `typescript/packages/mechanisms/moi/.prettierignore`

**Context.** Build output excluded.

```
docs/
dist/
node_modules/
coverage/
.github/
**/**/*.json
*.md
```

### 2.9 `typescript/packages/mechanisms/moi/README.md`

**Context.** The developer-facing document: why upfront, the three halves with install snippets, the in-process facilitator adapter (core's `getSupported` is sync on the facilitator and async on the client), the `errorReason` table mapping to spec rules, and how to run the tests.

```
# @x402/moi

x402 payment mechanism for MOI networks. Implements the `exact` scheme
using the `upfront`
payment flow, per [`specs/schemes/exact/scheme_exact_moi.md`]
(../../../../specs/schemes/exact/scheme_exact_moi.md).

## Why upfront

MOI uses the `upfront` flow by choice: the buyer settles a protocol-
native asset transfer from
its own account first, then presents a signed claim naming that
settled transfer. The spec
explains why this was chosen over the `authorization` and `escrow`
flows MOI also supports.

Two consequences. Core does not call `/verify` under this flow, so
every check lives in `settle`,
which confirms the transfer rather than executing one. And the buyer
paid its own fuel, so the
facilitator holds no keys, signs nothing, and can run in-process
inside the seller.

## Install

```bash
pnpm add @x402/moi
```
```

## Client

```ts
import { createMoiSigner } from "@x402/moi";
import { ExactMoiScheme } from "@x402/moi/exact/client";

const signer = await createMoiSigner(process.env.MOI_MNEMONIC, {
  rpcUrl: "https://dev.voyage-rpc.moi.technology/devnet/",
});
client.register("moi:devnet", new ExactMoiScheme(signer));
```

The signer is a js-moi-sdk wallet connected to a node, plus the account's participant identifier
and compressed public key. The buyer's transfer is submitted through that wallet; the claim is
signed with key 0, the key the identifier derives from. `createPaymentPayload` waits for the
transfer's receipt and refuses to build a claim over a transfer that was mined but did not
succeed.

## Server

```ts
import { ExactMoiScheme } from "@x402/moi/exact/server";

server.register("moi:devnet", new ExactMoiScheme());
```

Prices take three forms. `{ asset, amount }` in atomic units passes through. A suffixed string
such as `"0.10 KMOI"` converts with KMOI's nine decimals (1 KMOI = 1,000,000,000 anu). A bare
dollar price such as `"$0.10"` is refused, because MOI has no USD-pegged default asset, unless you
register a converter with `registerMoneyParser`. KMOI's decimals are built in; pass
`assetDecimals` for any other asset, or core will assume six when it displays an amount.

The server half also copies the resource URL into `extra.resource`. The buyer signs over it and
the facilitator checks it, which is what binds a payment to one request.

## Facilitator

```ts
import { createMoiChainReader, ExactMoiScheme } from
"@x402/moi/exact/facilitator";

const reader = createMoiChainReader({ rpcUrl: "https://dev.voyage-
rpc.moi.technology/devnet/" });
facilitator.register("moi:devnet", new ExactMoiScheme(reader,
{ spentStore }));
```

`createMoiChainReader` builds the three reads the facilitator needs
over a JSON-RPC node, using
js-moi-sdk: ECDSA verification through the SDK's `Signer.verify`,
participant-identifier
derivation matching `Wallet.getIdentifier` (a v0 tag, the 24 bytes of
the compressed key after
its parity byte, variant 0), and the transfer read-back described
below. Anything exposing
`getInteractionByHash` and `getInteractionReceipt` can stand in for
the node through the
`provider` option. Sub-accounts (identifiers with a non-zero variant)
are not derived; a claim
from one fails rule 3.

`spentStore` is the replay boundary and MUST be durable in production.
Its `reserve` has to be
atomic — under this flow two concurrent settlements can both observe
the same settled transfer
before either records it, so a separate has-then-add would let one
payment buy twice. The bundled
`InMemorySpentStore` is for development only.

### In-process facilitation

`x402ResourceServer` takes a `FacilitatorClient`, whose
`getSupported()` returns a promise;
`x402Facilitator.getSupported()` is synchronous, so wrap it to run
both in one process:

```ts
import { x402Facilitator } from "@x402/core/facilitator";
import { x402ResourceServer } from "@x402/core/server";

const facilitator = new x402Facilitator().register("moi:devnet", new
ExactMoiScheme(reader));
const local = {
  verify: (p, r) => facilitator.verify(p, r),
  settle: (p, r) => facilitator.settle(p, r),
  getSupported: async () => facilitator.getSupported(),
```

```
};
const server = new x402ResourceServer(local);
```

On the client, `wrapFetchWithPayment` comes from `@x402/fetch`, not `@x402/core`.

### What `settle` rejects

`errorReason` names the rule that failed:

| Reason | Rule |
| --- | --- |
| `moi_exact_invalid_payload` | claim, signature or public key missing; `keyId` not 0; key does not derive to `claim.from` |
| `moi_exact_invalid_signature` | ECDSA over the canonical claim bytes failed |
| `moi_exact_requirements_mismatch` | network, asset, payee, amount, or resource disagree with the quote |
| `moi_exact_payment_expired` | outside `[validAfter, validBefore]`, or a window wider than `maxTimeoutSeconds` |
| `moi_exact_transfer_not_found` | the node has no interaction at `claim.txHash` |
| `moi_exact_chain_read_failed` | the node could not be read, or the interaction is not a transfer |
| `moi_exact_transfer_mismatch` | the on-chain transfer disagrees with the claim (sender, beneficiary, asset, callsite, amount, or status) |
| `moi_exact_already_spent` | this transfer hash was settled before |

## Reading a transfer back

The receipt carries the status but neither the beneficiary nor the amount, so a full read takes
both `moi.InteractionByHash` and `moi.InteractionReceipt`. A transfer is an `ASSET_INVOKE`
operation (type `5`) routed to the `Transfer` callsite; the beneficiary and amount come from
POLO-decoding its calldata (`decodeTransferCalldata`). A refused transfer still returns an
interaction hash and a mined receipt, so the interaction-level `status` has to be checked rather
than assumed.

## Testing

```bash
pnpm test              # unit
pnpm test:integration  # against a live network
```

```
The integration file has two parts. The read-back tests need only
network access: they confirm
a transfer that already settled on Voyage devnet comes back through
the reader with the shape
rule 7 describes. The round trip pays, proves, and settles for real;
it runs only when
`CLIENT_MOI_MNEMONIC` (a funded buyer) and `SERVER_MOI_ADDRESS` are
set, and honours
`MOI_RPC_URL`, `MOI_ASSET_ID` (default: KMOI) and `MOI_AMOUNT`
(default: 1 atomic unit).
```

### 2.10 `typescript/packages/mechanisms/moi/CHANGELOG.md`

**Context.** Placeholder; Changesets writes the real entry on release.

```
# @x402/moi

## 2.25.0

Initial release. Implements the `exact` scheme on MOI networks
(`moi:*`) using the `upfront`
payment flow, per `specs/schemes/exact/scheme_exact_moi.md`. Ships
client, server and
facilitator halves, a mnemonic-based client signer, and a JSON-RPC
chain reader for the
facilitator.
```

### 2.11 `typescript/packages/mechanisms/moi/src/index.ts`

**Context.** Root entry point. Re-exports the client scheme as `ExactMoiScheme` plus every shared module, matching Stellar's root export shape.

```
export { ExactMoiScheme } from "./exact/index.js";
export * from "./constants.js";
export * from "./reader.js";
export * from "./shared.js";
export * from "./signer.js";
export * from "./types.js";
export * from "./utils.js";
export * from "./defaultAssets.js";
```

### 2.12 `typescript/packages/mechanisms/moi/src/constants.ts`

**Context.** The namespace, the devnet CAIP-2 id, and KMOI's post-0.13.0 asset id, decimals and symbol. The CAIP-2 string here follows whatever PR 1 decides on the reference format.

```
import type { Network } from "@x402/core/types";

/**
 * CAIP-2 identifiers for MOI networks.
```

```
 *
 * The reference form follows the `moi` namespace profile filed with
the Chain Agnostic
 * Standards Alliance (ChainAgnostic/namespaces, `moi/caip2`).
 */
export const MOI_NAMESPACE = "moi";

/** Voyage devnet. The only live public MOI network. */
export const MOI_DEVNET_CAIP2 = "moi:devnet" as Network;

/**
 * KMOI, the native token. Its identifier derives from its symbol and
standard (MASN), not from a
 * network, so it is the same on every MOI network. This is the post-
v0.13.0 identifier.
 */
export const KMOI_ASSET_ID =
"0x1080fffe4cd973c4eb83cdb8870c0de209736270491b7acc99873da100000000";

/** KMOI's decimals: 1 KMOI = 1,000,000,000 anu. Matches js-moi-
constants `KMOI_DECIMALS`. */
export const KMOI_DECIMALS = 9;

/** KMOI's ticker, for suffixed prices such as `"0.10 KMOI"`. */
export const KMOI_SYMBOL = "KMOI";

/** Matches any MOI network. The facilitator declares this as its CAIP
family. */
export const MOI_WILDCARD_CAIP2 = "moi:*";

/** The scheme this package implements. */
export const MOI_SCHEME = "exact";

/**
 * The callsite a native-asset transfer invokes (MAS0, and MASN for
KMOI).
 *
 * MOI has no dedicated transfer opcode. A transfer is an ASSET_INVOKE
operation routed to the
 * asset's `Transfer` routine by name, so the callsite is what
identifies it.
 */
export const TRANSFER_CALLSITE = "Transfer";

/** Operation type for ASSET_INVOKE, per js-moi-sdk's OpType enum. */
export const OP_ASSET_INVOKE = 5;

/** Interaction-level receipt status meaning the interaction
succeeded. */
```

```
export const RECEIPT_STATUS_OK = 0;

/** Default JSON-RPC endpoint for Voyage devnet. */
export const MOI_DEVNET_RPC_URL = "https://dev.voyage-
rpc.moi.technology/devnet/";

/** Seconds a claim stays valid when the requirements do not say. */
export const DEFAULT_CLAIM_TIMEOUT_SECONDS = 60;
```

### 2.13 `typescript/packages/mechanisms/moi/src/types.ts`

> **Context.** The claim the buyer signs (spec section PAYMENT-SIGNATURE) and the payload envelope. `network` was added to the claim on 10 September.

```
/**
 * The statement a buyer signs after paying.
 *
 * MOI settles before it proves, so this claim names a transfer that
already happened and binds
 * it to one request. Without the binding, a transfer hash lifted off
the public chain could be
 * redeemed by anyone watching.
 */
export interface MoiPaymentClaim {
  /** The paying account's participant identifier. */
  from: string;
  /** CAIP-2 network the transfer settled on. Must equal
`requirements.network`. */
  network: string;
  /** The payee. Must equal `requirements.payTo`. */
  to: string;
  /** Asset identifier (MAS0, or MASN for KMOI). */
  asset: string;
  /** Atomic units, decimal string. */
  value: string;
  /** Interaction hash of the transfer the buyer already submitted. */
  txHash: string;
  /** URL of the resource being bought. Binds this payment to one
request. */
  resource: string;
  /** Unix seconds. */
  validAfter: string;
  /** Unix seconds. */
  validBefore: string;
  /** 32 random bytes. Distinguishes two claims over the same
transfer. */
  nonce: string;
}

/** What travels in `PaymentPayload.payload` for this mechanism. */
```

```
export interface MoiPaymentPayload extends Record<string, unknown> {
  /** Compressed public key, hex, no 0x prefix. */
  publicKey: string;
  /** Which of the account's keys signed. MUST be 0: the key the
identifier derives from. */
  keyId: number;
  /** ECDSA secp256k1 over the canonical claim bytes. */
  signature: string;
  claim: MoiPaymentClaim;
}
```

### 2.14 `typescript/packages/mechanisms/moi/src/utils.ts`

> **Context.** Identifier comparison (case and `0x` insensitive), a network predicate, and `toBigInt`, which accepts both the hex the RPC returns and the decimal strings payloads carry. All quantities are bigint; the earlier draft lost precision through `Number()`.

```
import { MOI_NAMESPACE } from "./constants.js";

/**
 * Is this CAIP-2 identifier a MOI network?
 *
 * @param network - The identifier to test.
 * @returns True when the namespace is `moi`.
 */
export const isMoiNetwork = (network: string): boolean =>
network.startsWith(`${MOI_NAMESPACE}:`);

/**
 * Compare two MOI identifiers.
 *
 * Participant and asset identifiers arrive from several sources — a
payload, an RPC response,
 * configuration — and differ only in case or 0x prefix. Comparing
them raw produces false
 * mismatches, so every comparison in this package goes through here.
 *
 * @param a - First identifier.
 * @param b - Second identifier.
 * @returns True when they name the same thing.
 */
export const sameId = (a: string | undefined, b: string | undefined):
boolean =>
  normalizeId(a) === normalizeId(b) && normalizeId(a) !== "";

/**
 * Lower-case an identifier and drop its 0x prefix.
 *
 * @param value - The identifier.
```

```
 * @returns The normalized form, or an empty string when absent.
 */
export const normalizeId = (value: string | undefined): string =>
  (value ?? "").trim().toLowerCase().replace(/^0x/, "");

/**
 * Parse a hex or decimal quantity into a bigint.
 *
 * RPC responses return quantities as hex strings while payloads carry
decimal strings. Both
 * reach the same comparisons, and Number() would lose precision above
2^53, so both parse to
 * bigint here.
 *
 * @param value - Hex (0x-prefixed) or decimal string, or a number.
 * @returns The value as a bigint.
 */
export function toBigInt(value: string | number | bigint): bigint {
  if (typeof value === "bigint") return value;
  if (typeof value === "number") return BigInt(value);
  const trimmed = value.trim();
  return trimmed.startsWith("0x") || trimmed.startsWith("0X")
    ? BigInt(trimmed)
    : BigInt(trimmed === "" ? "0" : trimmed);
}
```

### 2.15 `typescript/packages/mechanisms/moi/src/shared.ts`

> **Context.** The canonical claim bytes from the spec: positional array, domain separator, `JSON.stringify` encoding. Used by both the client (to sign) and the facilitator (to verify). Also `nowSeconds` and the 32-byte nonce.

```
import type { MoiPaymentClaim } from "./types.js";

/**
 * Canonical bytes for signing a claim.
 *
 * Signer and verifier have to produce identical bytes or the
signature check fails, so the
 * fields are pinned positionally rather than left to JSON key
ordering. The first element is a
 * domain separator: it stops a signature made here being
reinterpreted as a signature over
 * anything else the same key might sign.
 *
 * @param claim - The claim to serialize.
 * @returns The canonical JSON string.
 */
export const canonicalClaim = (claim: MoiPaymentClaim): string =>
  JSON.stringify([
```

```
    "moi-x402-payment-claim-v1",
    claim.from,
    claim.network,
    claim.to,
    claim.asset,
    claim.value,
    claim.txHash,
    claim.resource,
    claim.validAfter,
    claim.validBefore,
    claim.nonce,
  ]);

/**
 * UTF-8 encoding of the canonical claim, which is what actually gets
signed.
 *
 * @param claim - The claim to serialize.
 * @returns The bytes to sign.
 */
export const canonicalClaimBytes = (claim: MoiPaymentClaim):
Uint8Array =>
  new TextEncoder().encode(canonicalClaim(claim));

/**
 * Current time in whole seconds.
 *
 * @returns Unix seconds.
 */
export const nowSeconds = (): number => Math.floor(Date.now() / 1000);

/**
 * A fresh 32-byte nonce.
 *
 * @returns Hex string, 0x-prefixed.
 */
export function randomNonce(): string {
  const bytes = new Uint8Array(32);
  globalThis.crypto.getRandomValues(bytes);
  return "0x" + Array.from(bytes, byte =>
byte.toString(16).padStart(2, "0")).join("");
}
```

### 2.16 `typescript/packages/mechanisms/moi/src/signer.ts`

**Context.** The two interfaces the halves depend on: `ClientMoiSigner` (a connected js-moi-sdk wallet, its identifier and compressed key) and `MoiChainReader` (the three reads a facilitator performs; no key material). `createMoiSigner(mnemonic,`

`{ rpcUrl | provider }`) connects the wallet to a node, which the earlier draft forgot.

```
import { JsonRpcProvider, Wallet } from "js-moi-sdk";
import { MOI_DEVNET_RPC_URL } from "./constants.js";
import type { AbstractProvider } from "js-moi-sdk";
/**
 * Signer shapes this mechanism needs.
 *
 * Kept narrow on purpose. A buyer needs to sign a claim and submit a
transfer; a facilitator
 * needs neither, because it only reads.
 */

/** What the buyer half needs from a MOI wallet. */
export interface ClientMoiSigner {
  /** The paying account's participant identifier, 0x-prefixed. */
  address: string;
  /** Compressed public key, hex, no 0x prefix. */
  publicKey: string;
  /** Which of the account's keys signs. Always 0: the identifier
derives from key 0. */
  keyId: number;
  /** A js-moi-sdk wallet, connected to a node, used to sign the claim
and submit the transfer. */
  wallet: {
    signingAlgorithms: { ecdsa_secp256k1: unknown };
    sign(bytes: Uint8Array, keyId: number, algorithm: unknown):
Promise<string>;
  };
}

/**
 * Chain reads the facilitator performs.
 *
 * No key material appears here. Under the upfront flow the
facilitator confirms a transfer that
 * already settled, so every check it makes is a read.
`createMoiChainReader` builds one over a
 * JSON-RPC node.
 */
export interface MoiChainReader {
  /**
   * Verify an ECDSA signature over bytes for a given public key.
   *
   * @param bytes - The signed message.
   * @param signature - The signature to check.
   * @param publicKey - Compressed public key, hex.
   * @returns True when the signature verifies.
   */
```

```
  verifySignature(bytes: Uint8Array, signature: string, publicKey:
string): boolean;

  /**
   * Derive a participant identifier from a public key.
   *
   * @param publicKey - Compressed public key, hex.
   * @returns The participant identifier.
   */
  identifierFromPublicKey(publicKey: string): string;

  /**
   * Read a settled transfer back off the chain.
   *
   * Both objects are needed: the receipt carries the status but
neither the beneficiary nor the
   * amount, and the interaction carries the operation but not whether
it succeeded.
   *
   * @param txHash - Interaction hash of the transfer.
   * @param assetId - The quoted asset, so a reader can pick the right
operation when an
   *   interaction carries several.
   * @returns What the chain says about it, or null when the
interaction does not exist.
   */
  readTransfer(txHash: string, assetId?: string):
Promise<SettledTransferFacts | null>;
}

/** The facts a facilitator compares a claim against. */
export interface SettledTransferFacts {
  /** `interaction.sender.id`. */
  sender: string;
  /** Beneficiary decoded from the operation's POLO calldata. */
  beneficiary: string;
  /** Amount decoded from the operation's POLO calldata. */
  amount: bigint;
  /** `payload.asset_id` of the ASSET_INVOKE operation. */
  assetId: string;
  /** `payload.callsite`. A transfer routes to `Transfer`. */
  callsite: string;
  /** True when the interaction-level receipt status is 0. */
  succeeded: boolean;
}

/** Options for {@link createMoiSigner}. */
export interface MoiSignerOptions {
  /** JSON-RPC endpoint the wallet submits to. Defaults to Voyage
devnet. Ignored when `provider` is given. */
```

```
  rpcUrl?: string;
  /** A js-moi-sdk provider to connect the wallet to. */
  provider?: AbstractProvider;
  /** HD derivation path. The SDK default is used when omitted. */
  derivationPath?: string;
}

/**
 * Build a client signer from a mnemonic.
 *
 * The derivation path selects the account: MOI derives a participant
identifier from the key
 * itself, so two paths on one mnemonic are two unrelated accounts as
far as the chain is
 * concerned. The wallet is connected to a node here because the buyer
half submits a transfer
 * through it; an unconnected wallet cannot send.
 *
 * @param mnemonic - BIP-39 mnemonic.
 * @param options - Endpoint or provider, and derivation path.
 * @returns A signer the client scheme can use.
 */
export async function createMoiSigner(
  mnemonic: string,
  options: MoiSignerOptions = {},
): Promise<ClientMoiSigner> {
  const wallet = await Wallet.fromMnemonic(mnemonic,
options.derivationPath);
  wallet.connect(options.provider ?? new
JsonRpcProvider(options.rpcUrl ?? MOI_DEVNET_RPC_URL));
  return {
    address: (await wallet.getIdentifier()).toHex(),
    publicKey: wallet.publicKey.replace(/^0x/, ""),
    keyId: 0,
    wallet: wallet as unknown as ClientMoiSigner["wallet"],
  };
}
```

### 2.17 `typescript/packages/mechanisms/moi/src/reader.ts`

**Context.** The default `MoiChainReader` over a JSON-RPC node. Signature verification goes through the SDK's own `Signer.verify`; identifier derivation mirrors `Wallet.getIdentifier`; `readTransfer(txHash, asset)` does both RPC reads and POLO-decodes the `Transfer` calldata. A missing interaction reads as `null`; other failures propagate so the facilitator can return `moi_exact_chain_read_failed`.

```
import {
  bytesToHex,
  createParticipantId,
```

```
  hexToBytes,
  JsonRpcProvider,
  ParticipantTagV0,
  Signer,
} from "js-moi-sdk";
import { Depolorizer } from "js-polo";
import {
  TRANSFER_CALLSITE,
  MOI_DEVNET_RPC_URL,
  OP_ASSET_INVOKE,
  RECEIPT_STATUS_OK,
} from "./constants.js";
import { sameId, toBigInt } from "./utils.js";
import type { MoiChainReader, SettledTransferFacts } from
"./signer.js";

/**
 * The two reads a facilitator needs from a node.
 *
 * `JsonRpcProvider` from js-moi-sdk satisfies this. Tests hand in a
stub.
 */
export interface MoiRpcReader {
  /**
   * `moi.InteractionByHash`.
   *
   * @param hash - The interaction hash.
   * @returns The interaction as the node returns it.
   */
  getInteractionByHash(hash: string): Promise<RpcInteraction>;
  /**
   * `moi.InteractionReceipt`.
   *
   * @param hash - The interaction hash.
   * @returns The receipt as the node returns it.
   */
  getInteractionReceipt(hash: string): Promise<RpcReceipt>;
}

/** The parts of `moi.InteractionByHash` this reader looks at. */
export interface RpcInteraction {
  sender: { id: string };
  ix_operations: { type: number; payload: unknown }[];
}

/** The part of `moi.InteractionReceipt` this reader looks at. */
export interface RpcReceipt {
  status: number;
}
```

```
/** Payload of an `ASSET_INVOKE` operation, as the node returns it. */
interface AssetInvokePayload {
  asset_id: string;
  callsite: string;
  calldata: string;
}

/** Options for {@link createMoiChainReader}. */
export interface MoiChainReaderOptions {
  /** JSON-RPC endpoint. Defaults to Voyage devnet. Ignored when
`provider` is given. */
  rpcUrl?: string;
  /** A js-moi-sdk provider, or anything else that performs the two
reads. */
  provider?: MoiRpcReader;
}

/**
 * A signer that can only verify.
 *
 * js-moi-sdk puts signature verification on its abstract `Signer`, so
verifying without a key
 * means subclassing it with every signing member refusing to run.
 */
class VerifyOnlySigner extends Signer {
  /** Never connects; nothing here talks to a node. */
  connect(): void {}

  /**
   * Key index the SDK asks for before signing.
   *
   * @returns Always 0; there is no key.
   */
  async getKeyId(): Promise<number> {
    return 0;
  }

  /**
   * Refuses: this signer has no account.
   *
   * @returns Never; there is no key to derive an identifier from.
   */
  async getIdentifier(): Promise<never> {
    throw new Error("VerifyOnlySigner holds no key");
  }

  /**
   * Refuses: this signer cannot sign.
   *
```

```
   * @returns Never; there is no key to sign with.
   */
  async sign(): Promise<never> {
    throw new Error("VerifyOnlySigner holds no key");
  }

  /**
   * Whether key material is loaded.
   *
   * @returns False; there is no key material to initialize.
   */
  isInitialized(): boolean {
    return false;
  }

  /**
   * Refuses: this signer cannot sign an interaction.
   *
   * @returns Never; there is no key to sign with.
   */
  async signInteraction(): Promise<never> {
    throw new Error("VerifyOnlySigner holds no key");
  }
}

/**
 * A {@link MoiChainReader} backed by a MOI JSON-RPC node.
 *
 * Holds no key material. Signature checks run in process, identifier
derivation is pure, and
 * the transfer read-back is two RPC calls plus a POLO decode.
 */
export class JsonRpcMoiChainReader implements MoiChainReader {
  private readonly verifier = new VerifyOnlySigner();

  /**
   * Wrap a node.
   *
   * @param rpc - The node, or a stub performing the two reads.
   */
  constructor(private readonly rpc: MoiRpcReader) {}

  /**
   * Verify an ECDSA secp256k1 signature over bytes.
   *
   * @param bytes - The signed message.
   * @param signature - The signature the buyer produced with
`Wallet.sign`.
   * @param publicKey - Compressed public key, hex, with or without
```

```
0x.
   * @returns True when the signature verifies.
   */
  verifySignature(bytes: Uint8Array, signature: string, publicKey:
string): boolean {
    return this.verifier.verify(bytes, signature, publicKey);
  }

  /**
   * Derive the participant identifier a compressed public key
controls.
   *
   * Mirrors `Wallet.getIdentifier` in js-moi-sdk: a v0 participant
tag, the 24 bytes of the key
   * after its parity byte as the fingerprint, and variant 0. Sub-
accounts (variant > 0) are not
   * derived here; a claim from one fails rule 3.
   *
   * @param publicKey - Compressed public key, hex, with or without
0x.
   * @returns The participant identifier, 0x-prefixed.
   */
  identifierFromPublicKey(publicKey: string): string {
    const key = hexToBytes(publicKey);
    if (key.length !== 33) {
      throw new Error(`expected a 33-byte compressed public key, got $
{key.length} bytes`);
    }
    return createParticipantId({
      tag: ParticipantTagV0,
      fingerprint: key.slice(1, 25),
      variant: 0,
    }).toHex();
  }

  /**
   * Read a settled transfer back off the chain.
   *
   * @param txHash - Interaction hash of the transfer.
   * @param assetId - The quoted asset, used to pick the operation
when an interaction carries
   *   several. When omitted, the first `Transfer` wins.
   * @returns What the chain says, or null when the interaction does
not exist.
   */
  async readTransfer(txHash: string, assetId?: string):
Promise<SettledTransferFacts | null> {
    let interaction: RpcInteraction;
    let receipt: RpcReceipt;
    try {
```

```
      [interaction, receipt] = await Promise.all([
        this.rpc.getInteractionByHash(txHash),
        this.rpc.getInteractionReceipt(txHash),
      ]);
    } catch (error) {
      if (isNotFound(error)) return null;
      throw error;
    }

    const invokes = (interaction.ix_operations ?? [])
      .filter(op => op.type === OP_ASSET_INVOKE)
      .map(op => op.payload as AssetInvokePayload);
    if (invokes.length === 0) {
      throw new Error(`interaction ${txHash} carries no ASSET_INVOKE
operation`);
    }
    const payload =
      invokes.find(
        p =>
          p?.callsite === TRANSFER_CALLSITE &&
          (assetId === undefined || sameId(p.asset_id, assetId)),
      ) ?? invokes[0];

    const { beneficiary, amount } =
decodeTransferCalldata(payload.calldata);
    return {
      sender: interaction.sender.id,
      beneficiary,
      amount,
      assetId: payload.asset_id,
      callsite: payload.callsite,
      succeeded: receipt.status === RECEIPT_STATUS_OK,
    };
  }
}

/**
 * Build a chain reader.
 *
 * @param options - An endpoint or a provider. Defaults to Voyage
devnet.
 * @returns A reader the facilitator scheme can use.
 */
export function createMoiChainReader(options: MoiChainReaderOptions =
{}): MoiChainReader {
  // js-moi-sdk 0.9.0-rc2 types `Interaction.sender` as a string; the
node returns
  // `{ id, sequence_id, key_id }`, which is what this reader reads.
The cast trusts the wire.
  const rpc =
```

```
    options.provider ??
    (new JsonRpcProvider(options.rpcUrl ?? MOI_DEVNET_RPC_URL) as
unknown as MoiRpcReader);
  return new JsonRpcMoiChainReader(rpc);
}

/**
 * Decode the calldata of a `Transfer` operation.
 *
 * The node POLO-encodes `TransferParams` as a document: `beneficiary`
is the 32-byte
 * identifier, `amount` an unsigned integer in atomic units.
 *
 * @param calldata - Hex, 0x-prefixed, as `moi.InteractionByHash`
returns it.
 * @returns The beneficiary (0x-prefixed) and amount.
 */
export function decodeTransferCalldata(calldata: string):
{ beneficiary: string; amount: bigint } {
  const document = new
Depolorizer(hexToBytes(calldata)).depolorizeDocument();
  return {
    beneficiary: "0x" + bytesToHex(document.getBytes("beneficiary")),
    amount: toBigInt(document.getInteger("amount")),
  };
}

/**
 * Does this error mean the node has no such interaction?
 *
 * Voyage devnet answers `moi.InteractionByHash` for an unknown hash
with "error fetching
 * interaction" and `moi.InteractionReceipt` with "tesseract hash not
found". Anything else —
 * a refused connection, a malformed response — is not a missing
transfer and propagates.
 *
 * @param error - What the provider threw.
 * @returns True when the interaction is absent.
 */
function isNotFound(error: unknown): boolean {
  const message = error instanceof Error ? error.message :
String(error);
  return /error fetching interaction|not found/i.test(message);
}
```

### 2.18 `typescript/packages/mechanisms/moi/src/defaultAssets.ts`

> **Context.** KMOI as the one built-in default asset so core's display path uses nine decimals instead of assuming six, and so `"0.10 KMOI"` prices and spend controls work. There is no USD-pegged asset on MOI, so `"$0.10"` does not resolve here.

```
import { KMOI_ASSET_ID, KMOI_DECIMALS, KMOI_SYMBOL, MOI_DEVNET_CAIP2 }
from "./constants.js";
import { isMoiNetwork, sameId } from "./utils.js";
import type {
  DefaultAsset,
  DefaultAssetTable,
  FindDefaultAsset,
  GetDefaultAsset,
} from "@x402/core/types";

export type MoiDefaultAsset = DefaultAsset;

/**
 * KMOI is the one asset this package knows.
 *
 * Other mechanisms list a USD-pegged asset first so that `"$0.10"`
resolves. MOI has no such
 * asset, so this table exists for two other reasons:
`getAssetDecimals` (core otherwise assumes
 * 6 decimals when displaying an amount) and suffixed prices such as
`"0.10 KMOI"`.
 */
const KMOI: MoiDefaultAsset = {
  asset: KMOI_ASSET_ID,
  decimals: KMOI_DECIMALS,
  symbol: KMOI_SYMBOL,
};

/** Default assets by CAIP-2 network. KMOI's identifier is network-
independent. */
export const DEFAULT_ASSETS: DefaultAssetTable<MoiDefaultAsset> = {
  [MOI_DEVNET_CAIP2]: [KMOI],
};

const assetsFor = (network: string): readonly MoiDefaultAsset[] =>
  DEFAULT_ASSETS[network] ?? (isMoiNetwork(network) ? [KMOI] : []);

/**
 * Look up a default asset by network and ticker.
 *
 * @param network - CAIP-2 network.
 * @param symbol - Ticker; omit for the network default.
 * @returns The matching entry.
 * @throws When the network or ticker is unknown.
```

```
 */
export const getDefaultAsset: GetDefaultAsset<MoiDefaultAsset> =
(network, symbol?) => {
  const assets = assetsFor(network);
  if (assets.length === 0) {
    throw new Error(`No default asset configured for network $
{network}`);
  }
  if (!symbol) {
    return assets[0];
  }
  const wanted = symbol.toUpperCase();
  const match = assets.find(entry => entry.symbol.toUpperCase() ===
wanted);
  if (!match) {
    throw new Error(`No ${symbol} default asset configured for network
${network}`);
  }
  return match;
};

/**
 * Reverse lookup by asset identifier and network.
 *
 * @param asset - Asset identifier from payment requirements.
 * @param network - CAIP-2 network.
 * @returns The matching entry, or undefined.
 */
export const findDefaultAsset: FindDefaultAsset<MoiDefaultAsset> =
(asset, network) =>
  assetsFor(network).find(entry => sameId(entry.asset, asset));
```

### 2.19 `typescript/packages/mechanisms/moi/src/exact/index.ts`

**Context.** Re-export used by the root entry point.

```
export { ExactMoiScheme } from "./client/scheme.js";
```

### 2.20 `typescript/packages/mechanisms/moi/src/exact/client/index.ts`

**Context.** Entry point `@x402/moi/exact/client`.

```
export { ExactMoiScheme } from "./scheme.js";
export type { ExactMoiClientOptions } from "./scheme.js";
```

### 2.21 `typescript/packages/mechanisms/moi/src/exact/client/scheme.ts`

**Context.** The buyer half. Submits the transfer through the wallet (`MAS0AssetLogic.transfer`, which builds the same `ASSET_INVOKE`/`Transfer` operation for MASN assets), waits for the receipt and refuses to claim a transfer that

did not succeed, clamps the validity window to the quote, then signs the claim with key 0. `fuel_limit`/`fuel_price` are passed to the SDK only when set.

```
import { MAS0AssetLogic } from "js-moi-sdk";
import { DEFAULT_CLAIM_TIMEOUT_SECONDS, MOI_SCHEME,
RECEIPT_STATUS_OK } from "../../constants.js";
import { findDefaultAsset } from "../../defaultAssets.js";
import { canonicalClaimBytes, nowSeconds, randomNonce } from
"../../shared.js";
import { toBigInt } from "../../utils.js";
import type { ClientMoiSigner } from "../../signer.js";
import type { MoiPaymentClaim, MoiPaymentPayload } from
"../../types.js";
import type { PaymentPayload, PaymentRequirements, SchemeNetworkClient
} from "@x402/core/types";

/** Options for the buyer half. */
export interface ExactMoiClientOptions {
  /** Fuel limit for the transfer the buyer submits. Omit to use the
SDK default. */
  fuelLimit?: number;
  /**
   * Fuel price for the transfer, in anu. Omit to use the SDK default,
which tracks the network's
   * minimum (50 anu from moipod v0.13.0). No RPC exposes the node's
price limit, so a value below
   * it is only reported when the interaction is refused.
   */
  fuelPrice?: number;
}

/**
 * The buyer half of `exact` on MOI.
 *
 * Under the upfront flow the buyer settles first and proves
afterwards, so this class submits a
 * transfer from its own account, waits for its receipt, and then
signs a claim naming it. Nothing
 * is handed to a third party to submit; the scheme spec explains why
that flow was chosen.
 */
export class ExactMoiScheme implements SchemeNetworkClient {
  readonly scheme = MOI_SCHEME;

  /** Lets core's spend controls match a `"KMOI"` entry against the
quoted asset id. */
  readonly findDefaultAsset = findDefaultAsset;

  /**
   * Bind the buyer half to a paying account.
```

```
   *
   * @param signer - The paying account.
   * @param options - Transfer options.
   */
  constructor(
    private readonly signer: ClientMoiSigner,
    private readonly options: ExactMoiClientOptions = {},
  ) {}

  /**
   * Settle a transfer, wait for its receipt, then sign a claim naming
it.
   *
   * @param x402Version - Protocol version echoed back into the
payload.
   * @param paymentRequirements - What the seller quoted.
   * @returns The payload for the PAYMENT-SIGNATURE header.
   */
  async createPaymentPayload(
    x402Version: number,
    paymentRequirements: PaymentRequirements,
  ): Promise<Pick<PaymentPayload, "x402Version" | "payload">> {
    const resource = this.requireResource(paymentRequirements);
    const amount = toBigInt(paymentRequirements.amount);

    // `MAS0AssetLogic` builds the ASSET_INVOKE/Transfer operation for
any protocol-native asset,
    // KMOI's MASN standard included: the callsite and calldata are
the same.
    const asset = new MAS0AssetLogic(paymentRequirements.asset,
this.signer.wallet as never);
    const interaction = await
asset.transfer(paymentRequirements.payTo, amount).send({
      ...(this.options.fuelLimit !== undefined && { fuel_limit:
this.options.fuelLimit }),
      ...(this.options.fuelPrice !== undefined && { fuel_price:
this.options.fuelPrice }),
    });

    // A refused transfer still mines and still has a hash, so the
receipt is what says whether
    // anything moved. Claiming a reverted transfer would only be
rejected by the facilitator.
    const receipt = await interaction.wait();
    if (receipt.status !== RECEIPT_STATUS_OK) {
      throw new Error(
        `MOI transfer ${interaction.hash} was mined but did not
succeed (receipt status ${receipt.status})`,
      );
    }
```

```
    // The window has to fit inside what the seller quoted. A claim
whose validBefore runs past
    // maxTimeoutSeconds is rejected by the facilitator, so it is
clamped rather than sent.
    const timeout = paymentRequirements.maxTimeoutSeconds ||
DEFAULT_CLAIM_TIMEOUT_SECONDS;
    const validAfter = nowSeconds();

    const claim: MoiPaymentClaim = {
      from: this.signer.address,
      network: paymentRequirements.network,
      to: paymentRequirements.payTo,
      asset: paymentRequirements.asset,
      value: amount.toString(),
      txHash: interaction.hash,
      resource,
      validAfter: String(validAfter),
      validBefore: String(validAfter + timeout),
      nonce: randomNonce(),
    };

    const signature = await this.signer.wallet.sign(
      canonicalClaimBytes(claim),
      this.signer.keyId,
      this.signer.wallet.signingAlgorithms.ecdsa_secp256k1,
    );

    const payload: MoiPaymentPayload = {
      publicKey: this.signer.publicKey,
      keyId: this.signer.keyId,
      signature,
      claim,
    };

    return { x402Version, payload };
  }

  /**
   * Read the resource URL the seller put in `extra`.
   *
   * The claim binds a payment to one resource, and the facilitator
enforces that binding, so a
   * payload without it would settle nothing. `PaymentPayloadContext`
does not carry the
   * ResourceInfo, so the seller's scheme copies the URL into
`extra.resource` for the buyer to
   * sign over.
   *
```

```
   * @param paymentRequirements - What the seller quoted.
   * @returns The resource URL.
   */
  private requireResource(paymentRequirements: PaymentRequirements):
string {
    const resource = (paymentRequirements.extra as Record<string,
unknown> | undefined)?.resource;
    if (typeof resource !== "string" || resource === "") {
      throw new Error(
        "PaymentRequirements.extra.resource is missing. The MOI scheme
binds each payment to " +
          "one resource, and the seller's scheme populates this field.
Without it the " +
          "facilitator cannot check the binding and will reject the
settlement.",
      );
    }
    return resource;
  }
}
```

### 2.22 typescript/packages/mechanisms/moi/src/exact/server/index.ts

**Context.** Entry point `@x402/moi/exact/server`.

```
export { ExactMoiScheme } from "./scheme.js";
export type { ExactMoiServerOptions } from "./scheme.js";
```

### 2.23 typescript/packages/mechanisms/moi/src/exact/server/scheme.ts

**Context.** The seller half. Declares `paymentFlow: "upfront"` (core writes it to the wire), parses prices (`{ asset, amount }`, a `"0.10 KMOI"` string, or a registered money parser; bare dollars are refused), and copies the resource URL into `extra.resource` through `enrichPaymentRequiredResponse`, the only hook that sees the `ResourceInfo`. That copy is what makes rule 5 enforceable.

```
import { convertToTokenAmount, parseMoney } from "@x402/core/utils";
import { MOI_SCHEME } from "../../constants.js";
import { findDefaultAsset, getDefaultAsset } from
"../../defaultAssets.js";
import { isMoiNetwork } from "../../utils.js";
import type {
  AssetAmount,
  MoneyParser,
  Network,
  PaymentFlowConfig,
  PaymentRequirements,
  Price,
  SchemeNetworkServer,
  SchemePaymentRequiredContext,
} from "@x402/core/types";
```

```
/** Options for the seller half. */
export interface ExactMoiServerOptions {
  /**
   * Decimals per asset id, for assets this package does not know.
   *
   * KMOI's decimals are built in. MOI exposes no decimals on an
asset, so for any other asset
   * core would assume 6 when displaying an amount; supply the real
value here.
   */
  assetDecimals?: Record<string, number>;
}

/**
 * The seller half of `exact` on MOI.
 *
 * Two jobs. It declares the payment flow, which is what lets core
drive an upfront mechanism
 * without scheme-specific knowledge. And it copies the resource URL
into `extra` so the buyer
 * can sign over it, because the buyer's half never sees the
ResourceInfo.
 */
export class ExactMoiScheme implements SchemeNetworkServer {
  readonly scheme = MOI_SCHEME;

  /**
   * MOI offers one way to move a MAS0 asset, so there is no on-wire
choice of transfer method.
   * The interface documents `"default"` for exactly this case, and
core strips the key from the
   * wire when it is used.
   */
  readonly defaultAssetTransferMethod = "default";

  /**
   * Upfront only.
   *
   * The buyer submits its own transfer and proves it afterwards;
nothing is handed to a
   * facilitator to submit later. Core reads this and writes
`extra.paymentFlow` onto the wire
   * itself, since the flow is not `authorization`.
   */
  readonly paymentFlows = {
    default: { supported: ["upfront"], default: "upfront" },
  } as const satisfies Record<string, PaymentFlowConfig>;
```

```
  /** Fields this scheme adds to `extra` after the requirements are
first built. */
  readonly dynamicExtraFields = ["resource"];

  private readonly moneyParsers: MoneyParser[] = [];

  /**
   * Build the seller half.
   *
   * @param options - Server options.
   */
  constructor(private readonly options: ExactMoiServerOptions = {}) {}

  /**
   * Register a converter for money strings that name no asset, such
as `"$0.10"`.
   *
   * MOI has no USD-pegged default asset, so without a parser such
prices are refused. Parsers
   * run in registration order; the first non-null result wins.
   *
   * @param parser - Converts an amount on a network to an asset
amount, or null to pass.
   * @returns This scheme, for chaining.
   */
  registerMoneyParser(parser: MoneyParser): ExactMoiScheme {
    this.moneyParsers.push(parser);
    return this;
  }

  /**
   * Decimals for an asset: the caller's override, else the built-in
table (KMOI).
   *
   * @param asset - Asset identifier.
   * @param network - The network being quoted.
   * @returns The decimals, or undefined when unknown.
   */
  getAssetDecimals(asset: string, network: Network): number |
undefined {
    return this.options.assetDecimals?.[asset] ??
findDefaultAsset(asset, network)?.decimals;
  }

  /**
   * Turn a price into an asset amount.
   *
   * Three forms are accepted. `{ asset, amount }` in atomic units
passes through. A suffixed
```

```
   * money string such as `"0.10 KMOI"` converts with that asset's
decimals. A bare money string
   * such as `"$0.10"` goes to the registered money parsers, and is
refused when none accepts it.
   *
   * @param price - The configured price.
   * @param network - The network being quoted.
   * @returns The asset and atomic amount.
   */
  async parsePrice(price: Price, network: Network):
Promise<AssetAmount> {
    if (typeof price === "object" && price !== null && "amount" in
price) {
      if (!price.asset) {
        throw new Error(`Asset must be specified for an AssetAmount
price on ${network}`);
      }
      return { amount: price.amount, asset: price.asset, extra:
price.extra ?? {} };
    }

    const { amount, symbol } = parseMoney(price);
    if (symbol) {
      const asset = getDefaultAsset(network, symbol);
      return {
        amount: convertToTokenAmount(amount, asset.decimals),
        asset: asset.asset,
        extra: {},
      };
    }
    for (const parser of this.moneyParsers) {
      const result = await parser(amount, network);
      if (result !== null) {
        return result;
      }
    }
    throw new Error(
      `Cannot convert "${String(price)}" on ${network}: MOI has no
USD-pegged default asset. ` +
        'Quote { asset, amount } in atomic units, a suffixed price
such as "0.10 KMOI", or ' +
        "register a money parser.",
    );
  }

  /**
   * Reject requirements that do not name a MOI network.
   *
   * @param paymentRequirements - The requirements being built.
   * @returns The requirements, unchanged.
```

```
   */
  async enhancePaymentRequirements(
    paymentRequirements: PaymentRequirements,
  ): Promise<PaymentRequirements> {
    if (!isMoiNetwork(paymentRequirements.network)) {
      throw new Error(`${paymentRequirements.network} is not a MOI
network`);
    }
    return paymentRequirements;
  }

  /**
   * Copy the resource URL into `extra` so the buyer can sign over it.
   *
   * This hook is the only place a scheme sees the ResourceInfo. The
buyer needs the URL to build
   * its claim and the facilitator compares that claim against the
authoritative
   * `PaymentPayload.resource`, so without this step the binding
cannot be checked.
   *
   * @param context - The payment-required context, carrying the
ResourceInfo.
   * @returns Requirements with `extra.resource` populated.
   */
  async enrichPaymentRequiredResponse(
    context: SchemePaymentRequiredContext,
  ): Promise<PaymentRequirements[]> {
    const resource = context.resourceInfo?.url;
    return context.requirements.map(requirement =>
      requirement.scheme === this.scheme &&
isMoiNetwork(requirement.network) && resource
        ? { ...requirement, extra: { ...requirement.extra,
resource } }
        : requirement,
    );
  }
}
```

### 2.24 typescript/packages/mechanisms/moi/src/exact/facilitator/index.ts

**Context.** Entry point `@x402/moi/exact/facilitator`; also exports the reader and spent store.

```
export { ExactMoiScheme, InMemorySpentStore } from "./scheme.js";
export type { ExactMoiFacilitatorOptions, SpentStore } from
"./scheme.js";
export {
  createMoiChainReader,
  decodeTransferCalldata,
  JsonRpcMoiChainReader,
```

```
} from "../../reader.js";
export type { MoiChainReaderOptions, MoiRpcReader } from
"../../reader.js";
export type { MoiChainReader, SettledTransferFacts } from
"../../signer.js";
```

### 2.25 typescript/packages/mechanisms/moi/src/exact/facilitator/scheme.ts

**Context.** The facilitator half: spec rules 1 to 8 in order inside `settle`, each returning a namespaced `errorReason`. `verify` is present for interface completeness but core does not call it under upfront. `getSigners()` is empty and `getExtra()` returns `{ areFeesSponsored: false }`. `SpentStore.reserve` is the atomic replay boundary; `InMemorySpentStore` is for development only.

```
import { TRANSFER_CALLSITE, MOI_SCHEME, MOI_WILDCARD_CAIP2 } from
"../../constants.js";
import { canonicalClaimBytes, nowSeconds } from "../../shared.js";
import { sameId, toBigInt } from "../../utils.js";
import type { MoiChainReader, SettledTransferFacts } from
"../../signer.js";
import type { MoiPaymentClaim, MoiPaymentPayload } from
"../../types.js";
import type {
  PaymentPayload,
  PaymentRequirements,
  SchemeNetworkFacilitator,
  SettleResponse,
  VerifyResponse,
} from "@x402/core/types";

/**
 * Records which transfers have already bought something.
 *
 * This is the replay boundary. `reserve` has to be atomic: under the
upfront flow two concurrent
 * settlements can both read the same settled transfer off the chain
before either records it, so
 * a separate has-then-add would let one payment buy twice.
 */
export interface SpentStore {
  /**
   * Claim a transfer hash, if nobody else has.
   *
   * @param txHash - The transfer to claim.
   * @returns True when this caller claimed it, false when it was
already spent.
   */
  reserve(txHash: string): boolean | Promise<boolean>;
}
```

```
/** In-process spent store. Development only: it forgets everything on
restart. */
export class InMemorySpentStore implements SpentStore {
  private readonly seen = new Set<string>();

  /**
   * Claim a transfer hash, if nobody else has.
   *
   * @param txHash - The transfer to claim.
   * @returns True when this caller claimed it.
   */
  reserve(txHash: string): boolean {
    const key = txHash.toLowerCase();
    if (this.seen.has(key)) return false;
    this.seen.add(key);
    return true;
  }
}

/** Options for the facilitator half. */
export interface ExactMoiFacilitatorOptions {
  /** Where spent transfer hashes are recorded. Must be durable in
production. */
  spentStore?: SpentStore;
}

/**
 * The facilitator half of `exact` on MOI.
 *
 * Under the upfront flow core does not call `/verify` at all, so
every rule lives in `settle`.
 * The money moved before the seller saw the request; settlement
confirms it and marks it spent.
 */
export class ExactMoiScheme implements SchemeNetworkFacilitator {
  readonly scheme = MOI_SCHEME;

  /** Any MOI network. */
  readonly caipFamily = MOI_WILDCARD_CAIP2;

  /** The buyer paid its own fuel when it submitted the transfer. */
  readonly areFeesSponsored = false;

  private readonly spent: SpentStore;

  /**
   * Build the facilitator half over a chain reader.
   *
   * @param chain - Chain reads. No key material: this facilitator
```

```
only reads.
   * @param options - Facilitator options.
   */
  constructor(
    private readonly chain: MoiChainReader,
    options: ExactMoiFacilitatorOptions = {},
  ) {
    this.spent = options.spentStore ?? new InMemorySpentStore();
  }

  /**
   * Advertise that fees are not sponsored.
   *
   * `paymentFlow` is not set here. Core writes it onto the wire from
the server scheme's
   * declaration whenever the resolved flow is not `authorization`.
   *
   * @returns Extra fields for the wire, the same on every MOI
network.
   */
  getExtra(): Record<string, unknown> | undefined {
    return { areFeesSponsored: false };
  }

  /**
   * This facilitator signs nothing, so it has no signing addresses on
any network.
   *
   * @returns An empty list.
   */
  getSigners(): string[] {
    return [];
  }

  /**
   * Check a payment without recording it.
   *
   * Core does not call this under the upfront flow, where validity is
established by settle. It
   * is implemented so the mechanism stays usable through the generic
verify path and so tests
   * can exercise the rules without consuming a transfer.
   *
   * @param payload - The payment payload.
   * @param requirements - What the seller quoted.
   * @returns Whether the payment is valid.
   */
  async verify(
    payload: PaymentPayload,
    requirements: PaymentRequirements,
```

```
  ): Promise<VerifyResponse> {
    return this.check(payload, requirements);
  }

  /**
   * Confirm the transfer and mark it spent.
   *
   * Re-runs every rule rather than trusting an earlier verify, then
reserves the transfer hash
   * atomically so one payment cannot buy twice.
   *
   * @param payload - The payment payload.
   * @param requirements - What the seller quoted.
   * @returns The settlement result.
   */
  async settle(
    payload: PaymentPayload,
    requirements: PaymentRequirements,
  ): Promise<SettleResponse> {
    const claim = (payload?.payload as MoiPaymentPayload |
undefined)?.claim;
    const checked = await this.check(payload, requirements);

    if (!checked.isValid || !claim) {
      return {
        success: false,
        errorReason: checked.invalidReason ??
"moi_exact_invalid_payload",
        errorMessage: checked.invalidMessage,
        transaction: claim?.txHash ?? "",
        network: requirements.network,
      };
    }

    // Rule 8. Atomic: a has-then-add would leave a window for
concurrent settlements.
    if (!(await this.spent.reserve(claim.txHash))) {
      return {
        success: false,
        errorReason: "moi_exact_already_spent",
        errorMessage: "this transfer has already been redeemed",
        transaction: claim.txHash,
        network: requirements.network,
      };
    }

    return {
      success: true,
      payer: claim.from,
      transaction: claim.txHash,
```

```
      network: requirements.network,
      amount: claim.value,
    };
  }

  /**
   * Rules 1 to 7 of the scheme specification.
   *
   * @param payload - The payment payload.
   * @param requirements - What the seller quoted.
   * @returns Whether the payment satisfies every rule.
   */
  private async check(
    payload: PaymentPayload,
    requirements: PaymentRequirements,
  ): Promise<VerifyResponse> {
    const inner = payload?.payload as MoiPaymentPayload | undefined;
    const claim = inner?.claim;

    // 1. Shape.
    if (!claim || !inner?.signature || !inner?.publicKey) {
      return fail("moi_exact_invalid_payload", "missing claim,
signature or publicKey");
    }

    // 2. Signature over the canonical claim bytes.
    let signatureOk = false;
    try {
      signatureOk = this.chain.verifySignature(
        canonicalClaimBytes(claim),
        inner.signature,
        inner.publicKey,
      );
    } catch {
      signatureOk = false;
    }
    if (!signatureOk) {
      return fail("moi_exact_invalid_signature", "ECDSA over the
canonical claim failed");
    }

    // 3. The signing key controls the paying account. Without this, a
valid signature could
    //    still name someone else's transfer, which is how a lifted
hash would be redeemed.
    //    The identifier derives from key 0, so only key 0 can be
checked by derivation.
    if (inner.keyId !== 0) {
      return fail(
        "moi_exact_invalid_payload",
```

```
        `keyId ${String(inner.keyId)} is not the key the identifier
derives from; this scheme accepts key 0 only`,
      );
    }
    let derived: string;
    try {
      derived = this.chain.identifierFromPublicKey(inner.publicKey);
    } catch (error) {
      return fail("moi_exact_invalid_payload", `unusable publicKey: $
{describe(error)}`);
    }
    if (!sameId(derived, claim.from)) {
      return fail(
        "moi_exact_invalid_payload",
        `key derives to ${derived}, which cannot control ${claim.from}
`,
      );
    }

    // 4. The claim matches the quote.
    const mismatches = this.quoteMismatches(claim, requirements);
    if (mismatches.length > 0) {
      return fail("moi_exact_requirements_mismatch", `mismatch: $
{mismatches.join(", ")}`);
    }

    // 5. Resource binding. Signed but unenforced, a claim for one
resource would settle a
    //    request for another on the same seller.
    const resourceCheck = this.resourceMismatch(claim, payload);
    if (resourceCheck) return fail("moi_exact_requirements_mismatch",
resourceCheck);

    // 6. Freshness, and a window no wider than the seller quoted.
    const freshness = this.freshnessProblem(claim, requirements);
    if (freshness) return fail("moi_exact_payment_expired",
freshness);

    // 7. The transfer settled, read back off the chain rather than
believed. A read that fails
    //    outright — node down, malformed calldata — is a rejection,
not a crash: nothing was
    //    confirmed, so nothing is settled.
    let facts: Awaited<ReturnType<MoiChainReader["readTransfer"]>>;
    try {
      facts = await this.chain.readTransfer(claim.txHash,
requirements.asset);
    } catch (error) {
      return fail(
        "moi_exact_chain_read_failed",
```

```
        `could not read ${claim.txHash}: ${describe(error)}`,
      );
    }
    if (!facts) {
      return fail("moi_exact_transfer_not_found", `no interaction at $
{claim.txHash}`);
    }
    const problems = this.transferProblems(claim, requirements,
facts);
    if (problems.length > 0) {
      return fail(
        "moi_exact_transfer_mismatch",
        `on-chain transfer disagrees: ${problems.join(", ")}`,
      );
    }

    return { isValid: true, payer: claim.from };
  }

  /**
   * Rule 4: compare the claim against the quote.
   *
   * @param claim - The signed claim.
   * @param requirements - What the seller quoted.
   * @returns Field names that disagree.
   */
  private quoteMismatches(claim: MoiPaymentClaim, requirements:
PaymentRequirements): string[] {
    const mismatches: string[] = [];
    if (claim.network !== requirements.network)
mismatches.push("network");
    if (!sameId(claim.asset, requirements.asset))
mismatches.push("asset");
    if (!sameId(claim.to, requirements.payTo))
mismatches.push("payTo");
    try {
      if (toBigInt(claim.value) < toBigInt(requirements.amount))
mismatches.push("amount");
    } catch {
      mismatches.push("amount");
    }
    return mismatches;
  }

  /**
   * Rule 5: the claim names the resource being bought.
   *
   * @param claim - The signed claim.
   * @param payload - The payment payload, carrying the authoritative
ResourceInfo.
```

```
   * @returns A problem description, or null when the binding holds.
   */
  private resourceMismatch(claim: MoiPaymentClaim, payload:
PaymentPayload): string | null {
    const url = payload?.resource?.url;
    if (typeof url !== "string" || url === "") {
      // Fail closed. Skipping the comparison would leave the binding
unenforced.
      return "payload carries no resource to bind this payment to";
    }
    return claim.resource === url
      ? null
      : `claim names resource ${claim.resource}, request is for ${url}
`;
  }

  /**
   * Rule 6: the claim is current, and its window is no wider than
quoted.
   *
   * @param claim - The signed claim.
   * @param requirements - What the seller quoted.
   * @returns A problem description, or null when fresh.
   */
  private freshnessProblem(
    claim: MoiPaymentClaim,
    requirements: PaymentRequirements,
  ): string | null {
    const now = nowSeconds();
    const after = Number(claim.validAfter);
    const before = Number(claim.validBefore);
    if (!Number.isFinite(after) || !Number.isFinite(before)) return
"unparseable validity window";
    if (now < after) return "not valid yet";
    if (now > before) return `expired ${now - before}s ago`;
    if (requirements.maxTimeoutSeconds && before - after >
requirements.maxTimeoutSeconds) {
      return `validity window ${before - after}s exceeds the quoted $
{requirements.maxTimeoutSeconds}s`;
    }
    return null;
  }

  /**
   * Rule 7: what the chain says has to match the claim.
   *
   * @param claim - The signed claim.
   * @param requirements - What the seller quoted.
   * @param facts - What the chain returned.
   * @returns Problem descriptions.
```

```
   */
  private transferProblems(
    claim: MoiPaymentClaim,
    requirements: PaymentRequirements,
    facts: SettledTransferFacts,
  ): string[] {
    const problems: string[] = [];
    if (!facts.succeeded) problems.push("interaction reverted");
    if (!sameId(facts.sender, claim.from)) problems.push(`sender $
{facts.sender}`);
    if (!sameId(facts.beneficiary, claim.to))
problems.push(`beneficiary ${facts.beneficiary}`);
    if (!sameId(facts.assetId, requirements.asset))
problems.push(`asset ${facts.assetId}`);
    if (facts.callsite !== TRANSFER_CALLSITE) problems.push(`callsite
${facts.callsite}`);
    if (facts.amount < toBigInt(claim.value)) {
      problems.push(`amount ${facts.amount} < ${claim.value}`);
    }
    return problems;
  }
}

/**
 * Build a rejection.
 *
 * @param reason - Machine-readable reason.
 * @param message - Human-readable detail.
 * @returns An invalid VerifyResponse.
 */
const fail = (reason: string, message: string): VerifyResponse => ({
  isValid: false,
  invalidReason: reason,
  invalidMessage: message,
});

/**
 * One line for an error of unknown shape.
 *
 * @param error - Whatever was thrown.
 * @returns Its message.
 */
const describe = (error: unknown): string =>
  error instanceof Error ? error.message : String(error);
```

### 2.26 `typescript/packages/mechanisms/moi/test/unit/facilitator.test.ts`

**Context.** Nineteen tests: one per rule plus the failure modes that were bugs in the earlier draft (resource not checked, window widened, wrong callsite accepted, non-

atomic replay). Signatures are made with the SDK's own ECDSA, so the canonical bytes are checked end to end.

```
import { describe, expect, it, vi } from "vitest";
import { TRANSFER_CALLSITE } from "../../src/constants.js";
import { ExactMoiScheme, InMemorySpentStore } from
"../../src/exact/facilitator/scheme.js";
import { canonicalClaimBytes } from "../../src/shared.js";
import type { MoiChainReader } from "../../src/signer.js";
import type { MoiPaymentClaim } from "../../src/types.js";

const PAYER =
"0x00000000aaaa0000000000000000000000000000000000000000000000cccc";
const PAYEE =
"0x00000000bbbb0000000000000000000000000000000000000000000000dddd";
const ASSET =
"0x10800000eeee0000000000000000000000000000000000000000000000ff";
const RESOURCE = "https://api.example.com/forecast";

const claim = (over: Partial<MoiPaymentClaim> = {}): MoiPaymentClaim
=> {
  const now = Math.floor(Date.now() / 1000);
  return {
    from: PAYER,
    network: "moi:devnet",
    to: PAYEE,
    asset: ASSET,
    value: "1000",
    txHash: "0x" + "ab".repeat(32),
    resource: RESOURCE,
    validAfter: String(now - 1),
    validBefore: String(now + 59),
    nonce: "0x" + "11".repeat(32),
    ...over,
  };
};

// `null` means the payload carries no ResourceInfo at all. An
explicit `undefined` would hit
// the default parameter instead, which is how the fail-closed case
first slipped through.
const payload = (c: MoiPaymentClaim, resourceUrl: string | null =
RESOURCE) =>
  ({
    x402Version: 2,
    resource: resourceUrl === null ? undefined : { url: resourceUrl },
    accepted: {} as never,
    payload: { publicKey: "02aa", keyId: 0, signature: "0xsig", claim:
c },
  }) as never;
```

```
const requirements = (over: Record<string, unknown> = {}) =>
  ({
    scheme: "exact",
    network: "moi:devnet",
    asset: ASSET,
    amount: "1000",
    payTo: PAYEE,
    maxTimeoutSeconds: 60,
    extra: { resource: RESOURCE },
    ...over,
  }) as never;

const chain = (over: Partial<MoiChainReader> = {}): MoiChainReader =>
({
  verifySignature: () => true,
  identifierFromPublicKey: () => PAYER,
  readTransfer: async () => ({
    sender: PAYER,
    beneficiary: PAYEE,
    amount: 1000n,
    assetId: ASSET,
    callsite: TRANSFER_CALLSITE,
    succeeded: true,
  }),
  ...over,
});

describe("ExactMoiScheme facilitator", () => {
  it("settles a well-formed payment", async () => {
    const scheme = new ExactMoiScheme(chain());
    const result = await scheme.settle(payload(claim()),
requirements());
    expect(result.success).toBe(true);
    expect(result.payer).toBe(PAYER);
  });

  it("rejects a claim whose resource is not the resource being
bought", async () => {
    // The binding exists to stop a claim for a cheap resource
settling a request for another.
    const scheme = new ExactMoiScheme(chain());
    const result = await scheme.settle(
      payload(claim({ resource: "https://api.example.com/cheap" })),
      requirements(),
    );
    expect(result.success).toBe(false);
    expect(result.errorMessage).toMatch(/resource/i);
  });
```

```
  it("rejects a claim signed for a different network", async () => {
    // The buyer commits to the network in the signed bytes, so a
facilitator pointed at the
    // wrong RPC cannot settle a claim made elsewhere.
    const scheme = new ExactMoiScheme(chain());
    const result = await scheme.settle(payload(claim({ network:
"moi:other" })), requirements());
    expect(result.success).toBe(false);
    expect(result.errorMessage).toMatch(/network/i);
  });

  it("fails closed when the payload carries no resource", async () =>
{
    const scheme = new ExactMoiScheme(chain());
    const result = await scheme.settle(payload(claim(), null),
requirements());
    expect(result.success).toBe(false);
  });

  it("rejects a validity window wider than the seller quoted", async
() => {
    const now = Math.floor(Date.now() / 1000);
    const scheme = new ExactMoiScheme(chain());
    const result = await scheme.settle(
      payload(claim({ validAfter: String(now - 1), validBefore:
String(now + 600) })),
      requirements({ maxTimeoutSeconds: 60 }),
    );
    expect(result.success).toBe(false);
    expect(result.errorMessage).toMatch(/exceeds the quoted/);
  });

  it("rejects a signing key that does not control the paying account",
async () => {
    // A valid signature alone could still name someone else's
transfer.
    const scheme = new ExactMoiScheme(chain({ identifierFromPublicKey:
() => PAYEE }));
    const result = await scheme.settle(payload(claim()),
requirements());
    expect(result.success).toBe(false);
    expect(result.errorMessage).toMatch(/cannot control/);
  });

  it("rejects a claim signed with a key other than key 0", async () =>
{
    // The identifier derives from key 0; the account's other keys
cannot be checked by
```

```
    // derivation, so the spec pins keyId to 0.
    const scheme = new ExactMoiScheme(chain());
    const withKey = { ...(payload(claim()) as Record<string, unknown>)
};
    withKey.payload = { ...(withKey.payload as Record<string,
unknown>), keyId: 1 };
    const result = await scheme.settle(withKey as never,
requirements());
    expect(result.success).toBe(false);
    expect(result.errorMessage).toMatch(/keyId 1/);
  });

  it("rejects, rather than throws, when the chain cannot be read",
async () => {
    const scheme = new ExactMoiScheme(
      chain({
        readTransfer: async () => {
          throw new Error("fetch failed: ECONNREFUSED");
        },
      }),
    );
    const result = await scheme.settle(payload(claim()),
requirements());
    expect(result.success).toBe(false);
    expect(result.errorReason).toBe("moi_exact_chain_read_failed");
  });

  it("passes the quoted asset to the reader so it can pick the right
operation", async () => {
    const readTransfer = vi.fn(async () => ({
      sender: PAYER,
      beneficiary: PAYEE,
      amount: 1000n,
      assetId: ASSET,
      callsite: TRANSFER_CALLSITE,
      succeeded: true,
    }));
    const scheme = new ExactMoiScheme(chain({ readTransfer }));
    await scheme.settle(payload(claim()), requirements());
    expect(readTransfer).toHaveBeenCalledWith(claim().txHash, ASSET);
  });

  it("rejects a transfer that was mined but reverted", async () => {
    // A refused MAS0 transfer still returns an interaction hash.
    const scheme = new ExactMoiScheme(
      chain({
        readTransfer: async () => ({
          sender: PAYER,
          beneficiary: PAYEE,
          amount: 1000n,
```

```
          assetId: ASSET,
          callsite: TRANSFER_CALLSITE,
          succeeded: false,
        }),
      }),
    );
    const result = await scheme.settle(payload(claim()),
requirements());
    expect(result.success).toBe(false);
    expect(result.errorMessage).toMatch(/reverted/);
  });

  it("rejects an operation that is not a MAS0 Transfer", async () => {
    const scheme = new ExactMoiScheme(
      chain({
        readTransfer: async () => ({
          sender: PAYER,
          beneficiary: PAYEE,
          amount: 1000n,
          assetId: ASSET,
          callsite: "TransferFrom",
          succeeded: true,
        }),
      }),
    );
    const result = await scheme.settle(payload(claim()),
requirements());
    expect(result.success).toBe(false);
    expect(result.errorMessage).toMatch(/callsite/);
  });

  it("rejects an underpaying transfer and accepts an overpaying one",
async () => {
    const under = new ExactMoiScheme(
      chain({
        readTransfer: async () => ({
          sender: PAYER,
          beneficiary: PAYEE,
          amount: 999n,
          assetId: ASSET,
          callsite: TRANSFER_CALLSITE,
          succeeded: true,
        }),
      }),
    );
    expect((await under.settle(payload(claim()),
requirements())).success).toBe(false);

    const over = new ExactMoiScheme(
      chain({
```

```
        readTransfer: async () => ({
          sender: PAYER,
          beneficiary: PAYEE,
          amount: 5000n,
          assetId: ASSET,
          callsite: TRANSFER_CALLSITE,
          succeeded: true,
        }),
      }),
    );
    expect((await over.settle(payload(claim()),
requirements())).success).toBe(true);
  });

  it("spends a transfer once", async () => {
    const scheme = new ExactMoiScheme(chain());
    const first = await scheme.settle(payload(claim()),
requirements());
    const second = await scheme.settle(payload(claim()),
requirements());
    expect(first.success).toBe(true);
    expect(second.success).toBe(false);
    expect(second.errorReason).toBe("moi_exact_already_spent");
  });

  it("reserves atomically, so concurrent settlements cannot both win",
async () => {
    const scheme = new ExactMoiScheme(chain());
    const results = await Promise.all([
      scheme.settle(payload(claim()), requirements()),
      scheme.settle(payload(claim()), requirements()),
    ]);
    expect(results.filter(r => r.success)).toHaveLength(1);
  });

  it("signs nothing and sponsors nothing", () => {
    const scheme = new ExactMoiScheme(chain());
    expect(scheme.getSigners()).toEqual([]);
    expect(scheme.areFeesSponsored).toBe(false);
    // paymentFlow is core's to write, not the facilitator's.
    expect(scheme.getExtra()).toEqual({ areFeesSponsored: false });
    expect(scheme.getExtra()).not.toHaveProperty("paymentFlow");
  });

  it("checks the signature over the canonical claim bytes", async ()
=> {
    const verify = vi.fn().mockReturnValue(true);
    const scheme = new ExactMoiScheme(chain({ verifySignature:
verify }));
    const c = claim();
```

```
    await scheme.settle(payload(c), requirements());
    expect(verify).toHaveBeenCalledWith(canonicalClaimBytes(c),
"0xsig", "02aa");
  });
});

describe("InMemorySpentStore", () => {
  it("lets one caller claim a hash", () => {
    const store = new InMemorySpentStore();
    expect(store.reserve("0xAB")).toBe(true);
    expect(store.reserve("0xab")).toBe(false);
  });
});
```

### 2.27 `typescript/packages/mechanisms/moi/test/unit/reader.test.ts`

> **Context.** Decodes the POLO calldata of a real devnet transfer, exercises multi-operation interactions, missing interactions and RPC errors, and checks identifier derivation against what the wallet reports.

```
import { Wallet } from "js-moi-sdk";
import { describe, expect, it, vi } from "vitest";
import {
  createMoiChainReader,
  decodeTransferCalldata,
  JsonRpcMoiChainReader,
} from "../../src/reader.js";
import type { RpcInteraction, RpcReceipt } from "../../src/reader.js";

// A real KMOI transfer settled on Voyage devnet (sequence 0x4f of the
sender), captured on
// 2026-09-10. The calldata is the node's own POLO document, so the
decode below is against the
// wire, not a hand-made fixture.
const SETTLED = {
  hash:
"0x18f457917b15bcbfa16022b482a4f84be77a4a56f8be5f8ee88a0095d0b642b8",
  sender:
"0x00000000bfa45bc089945cf2befd0bd488e863562628cb47b14f3d1800000000",
  beneficiary:
"0x00000000ea3f6fdc706bd6ce6f5725f22e2231715f5794c782a470b300000000",
  asset:
"0x108000004cd973c4eb83cdb8870c0de209736270491b7acc99873da100000000",
  calldata:

"0x0d6f06658601b502616d6f756e7403fa62656e656669636961727906000000000ea3
f6fdc706bd6ce6f5725f22e2231715f5794c782a470b300000000",
  amount: 250n,
};
```

```
const interaction = (ops: RpcInteraction["ix_operations"]):
RpcInteraction => ({
  sender: { id: SETTLED.sender },
  ix_operations: ops,
});

const transferOp = (asset = SETTLED.asset, callsite = "Transfer") =>
({
  type: 5,
  payload: { asset_id: asset, callsite, calldata: SETTLED.calldata },
});

const rpc = (ix: RpcInteraction, receipt: RpcReceipt = { status: 0 })
=> ({
  getInteractionByHash: vi.fn(async () => ix),
  getInteractionReceipt: vi.fn(async () => receipt),
});

describe("decodeTransferCalldata", () => {
  it("decodes the node's POLO document to beneficiary and amount", ()
=> {
    expect(decodeTransferCalldata(SETTLED.calldata)).toEqual({
      beneficiary: SETTLED.beneficiary,
      amount: SETTLED.amount,
    });
  });
});

describe("JsonRpcMoiChainReader.readTransfer", () => {
  it("returns the facts the facilitator compares against", async () =>
{
    const reader = new
JsonRpcMoiChainReader(rpc(interaction([transferOp()])));
    await expect(reader.readTransfer(SETTLED.hash,
SETTLED.asset)).resolves.toEqual({
      sender: SETTLED.sender,
      beneficiary: SETTLED.beneficiary,
      amount: SETTLED.amount,
      assetId: SETTLED.asset,
      callsite: "Transfer",
      succeeded: true,
    });
  });

  it("reports a mined-but-refused transfer as not succeeded", async ()
=> {
    const reader = new
JsonRpcMoiChainReader(rpc(interaction([transferOp()]), { status:
1 })));
```

```
    const facts = await reader.readTransfer(SETTLED.hash,
SETTLED.asset);
    expect(facts?.succeeded).toBe(false);
  });

  it("picks the operation on the quoted asset when an interaction
carries several", async () => {
    const other = "0x1080fffe" + "11".repeat(28);
    const reader = new JsonRpcMoiChainReader(
      rpc(interaction([transferOp(other),
transferOp(SETTLED.asset)])),
    );
    const facts = await reader.readTransfer(SETTLED.hash,
SETTLED.asset);
    expect(facts?.assetId).toBe(SETTLED.asset);
  });

  it("falls back to the first asset operation so the facilitator sees
the mismatch", async () => {
    const reader = new JsonRpcMoiChainReader(
      rpc(interaction([transferOp(SETTLED.asset, "TransferFrom")])),
    );
    const facts = await reader.readTransfer(SETTLED.hash,
SETTLED.asset);
    expect(facts?.callsite).toBe("TransferFrom");
  });

  it("returns null when the node has no such interaction", async () =>
{
    const reader = new JsonRpcMoiChainReader({
      getInteractionByHash: async () => {
        throw new Error("error fetching interaction");
      },
      getInteractionReceipt: async () => {
        throw new Error("tesseract hash not found: key not found");
      },
    });
    await
expect(reader.readTransfer(SETTLED.hash)).resolves.toBeNull();
  });

  it("propagates errors that are not a missing interaction", async ()
=> {
    const reader = new JsonRpcMoiChainReader({
      getInteractionByHash: async () => {
        throw new Error("fetch failed: ECONNREFUSED");
      },
      getInteractionReceipt: async () => ({ status: 0 }),
    });
    await
```

```
expect(reader.readTransfer(SETTLED.hash)).rejects.toThrow(/ECONNREFUSE
D/);
  });

  it("refuses an interaction with no asset operation", async () => {
    const reader = new JsonRpcMoiChainReader(rpc(interaction([{ type:
12, payload: {} }])));
    await
expect(reader.readTransfer(SETTLED.hash)).rejects.toThrow(/no
ASSET_INVOKE/);
  });
});

describe("JsonRpcMoiChainReader keys", () => {
  it("derives the same identifier the wallet reports for its key",
async () => {
    const wallet = await Wallet.createRandom();
    const reader = createMoiChainReader({ provider:
rpc(interaction([])) });
    const expected = (await wallet.getIdentifier()).toHex();

expect(reader.identifierFromPublicKey(wallet.publicKey)).toBe(expected
);

expect(reader.identifierFromPublicKey(wallet.publicKey.replace(/^0x/,
"").toBe(expected);
  });

  it("rejects a key that is not a compressed secp256k1 point", () => {
    const reader = createMoiChainReader({ provider:
rpc(interaction([])) });
    expect(() => reader.identifierFromPublicKey("0x" +
"ab".repeat(20))).toThrow(/33-byte/);
  });

  it("verifies a wallet signature and rejects a tampered message",
async () => {
    const wallet = await Wallet.createRandom();
    const reader = createMoiChainReader({ provider:
rpc(interaction([])) });
    const message = new TextEncoder().encode('["moi-x402-payment-
claim-v1","a","b"]');
    const signature = await wallet.sign(message, 0,
wallet.signingAlgorithms.ecdsa_secp256k1);
    expect(reader.verifySignature(message, signature,
wallet.publicKey)).toBe(true);
    expect(
      reader.verifySignature(new TextEncoder().encode("tampered"),
signature, wallet.publicKey),
    ).toBe(false);
```

```
  });
});
```

### 2.28 `typescript/packages/mechanisms/moi/test/unit/server.test.ts`

**Context.** Eight tests for price parsing, decimals, the resource copy and network gating.

```
import { describe, expect, it } from "vitest";
import { KMOI_ASSET_ID } from "../../src/constants.js";
import { ExactMoiScheme } from "../../src/exact/server/scheme.js";

const NET = "moi:devnet" as never;
const ASSET =
"0x10800000eeee0000000000000000000000000000000000000000000000000000ff";

describe("ExactMoiScheme server", () => {
  it("declares the upfront flow and no on-wire transfer method", () =>
{
    const scheme = new ExactMoiScheme();
    expect(scheme.paymentFlows.default).toEqual({ supported:
["upfront"], default: "upfront" });
    expect(scheme.defaultAssetTransferMethod).toBe("default");
  });

  it("passes an atomic { asset, amount } price through", async () => {
    const scheme = new ExactMoiScheme();
    await expect(scheme.parsePrice({ asset: ASSET, amount: "1000" },
NET)).resolves.toEqual({
      asset: ASSET,
      amount: "1000",
      extra: {},
    });
  });

  it("converts a KMOI-suffixed price with nine decimals", async () =>
{
    const scheme = new ExactMoiScheme();
    await expect(scheme.parsePrice("0.10 KMOI",
NET)).resolves.toEqual({
      asset: KMOI_ASSET_ID,
      amount: "100000000",
      extra: {},
    });
  });

  it("refuses a dollar price when no money parser is registered",
async () => {
    const scheme = new ExactMoiScheme();
    await expect(scheme.parsePrice("$0.10",
```

```
NET)).rejects.toThrow(/money parser/);
  });

  it("uses a registered money parser for dollar prices", async () => {
    const scheme = new ExactMoiScheme().registerMoneyParser(async
amount => ({
      asset: ASSET,
      amount: String(Number(amount) * 1_000_000),
      extra: {},
    }));
    await expect(scheme.parsePrice("$0.10", NET)).resolves.toEqual({
      asset: ASSET,
      amount: "100000",
      extra: {},
    });
  });

  it("knows KMOI's decimals and takes overrides for other assets", ()
=> {
    const scheme = new ExactMoiScheme({ assetDecimals: { [ASSET]:
4 } });
    expect(scheme.getAssetDecimals(KMOI_ASSET_ID, NET)).toBe(9);
    expect(scheme.getAssetDecimals(ASSET, NET)).toBe(4);
    expect(new ExactMoiScheme().getAssetDecimals(ASSET,
NET)).toBeUndefined();
  });

  it("copies the resource URL into extra for MOI entries only", async
() => {
    const scheme = new ExactMoiScheme();
    const out = await scheme.enrichPaymentRequiredResponse({
      resourceInfo: { url: "https://api.example.com/forecast" },
      requirements: [
        { scheme: "exact", network: NET, extra: {} },
        { scheme: "exact", network: "eip155:8453", extra: {} },
      ],
    } as never);
    expect(out[0].extra).toEqual({ resource:
"https://api.example.com/forecast" });
    expect(out[1].extra).toEqual({});
  });

  it("rejects requirements on a non-MOI network", async () => {
    const scheme = new ExactMoiScheme();
    await expect(
      scheme.enhancePaymentRequirements({ network: "eip155:8453" } as
never),
    ).rejects.toThrow(/not a MOI network/);
  });
});
```

### 2.29 typescript/packages/mechanisms/moi/test/integrations/exact-moi.test.ts

> **Context.** Two parts. The read-back tests need only network access and pass against Voyage devnet today. The full pay-prove-settle round trip runs only with `CLIENT_MOI_MNEMONIC` and `SERVER_MOI_ADDRESS`; it has not yet been run with a funded wallet.

```
import { KMOI_ASSET_ID } from "js-moi-sdk";
import { describe, expect, it } from "vitest";
import { MOI_DEVNET_CAIP2, MOI_DEVNET_RPC_URL } from
"../../src/constants.js";
import { ExactMoiScheme as ClientScheme } from
"../../src/exact/client/scheme.js";
import { ExactMoiScheme as FacilitatorScheme } from
"../../src/exact/facilitator/scheme.js";
import { ExactMoiScheme as ServerScheme } from
"../../src/exact/server/scheme.js";
import { createMoiChainReader } from "../../src/reader.js";
import { createMoiSigner } from "../../src/signer.js";

/**
 * Against a live MOI network.
 *
 * The read-back half runs with nothing but network access: it
confirms a transfer that already
 * settled on Voyage devnet comes back through `moi.InteractionByHash`
and
 * `moi.InteractionReceipt` with the shape the spec's rule 7
describes, POLO calldata included.
 *
 * The round trip needs a funded buyer. It skips without
`CLIENT_MOI_MNEMONIC` and
 * `SERVER_MOI_ADDRESS`, so a checkout without credentials stays
green. When it runs, real value
 * moves: the buyer pays `MOI_AMOUNT` atomic units of `MOI_ASSET_ID`
to the seller, and the
 * facilitator settles that transfer exactly once.
 */
const RPC_URL = process.env.MOI_RPC_URL ?? MOI_DEVNET_RPC_URL;

// A KMOI transfer settled on Voyage devnet on 2026-09-10. If devnet
is reset, replace it with
// any settled `Transfer` and its decoded fields.
const SETTLED = {
  hash:
"0x18f457917b15bcbfa16022b482a4f84be77a4a56f8be5f8ee88a0095d0b642b8",
  sender:
"0x00000000bfa45bc089945cf2befd0bd488e863562628cb47b14f3d1800000000",
  beneficiary:
```

```
"0x00000000ea3f6fdc706bd6ce6f5725f22e2231715f5794c782a470b300000000",
  asset:
"0x108000004cd973c4eb83cdb8870c0de209736270491b7acc99873da100000000",
  amount: 250n,
};

describe("exact on MOI, live read-back", () => {
  it("reaches the configured node", async () => {
    const response = await fetch(RPC_URL, {
      method: "POST",
      headers: { "content-type": "application/json" },
      body: JSON.stringify({ jsonrpc: "2.0", id: 1, method:
"net.Version", params: [{}] }),
    });
    const body = (await response.json()) as { result?: string };
    expect(typeof body.result).toBe("string");
  });

  it("reads a settled transfer back as rule 7 describes", async () =>
{
    const reader = createMoiChainReader({ rpcUrl: RPC_URL });
    await expect(reader.readTransfer(SETTLED.hash,
SETTLED.asset)).resolves.toEqual({
      sender: SETTLED.sender,
      beneficiary: SETTLED.beneficiary,
      amount: SETTLED.amount,
      assetId: SETTLED.asset,
      callsite: "Transfer",
      succeeded: true,
    });
  });

  it("answers null for an interaction that does not exist", async ()
=> {
    const reader = createMoiChainReader({ rpcUrl: RPC_URL });
    await expect(reader.readTransfer("0x" +
"ab".repeat(32))).resolves.toBeNull();
  });
});

const MNEMONIC = process.env.CLIENT_MOI_MNEMONIC;
const PAY_TO = process.env.SERVER_MOI_ADDRESS;
const ASSET = process.env.MOI_ASSET_ID ?? KMOI_ASSET_ID;
const AMOUNT = process.env.MOI_AMOUNT ?? "1";

describe.skipIf(!MNEMONIC || !PAY_TO)("exact on MOI, live round trip",
() => {
  it("pays, proves, and settles exactly once", async () => {
    const signer = await createMoiSigner(MNEMONIC!, { rpcUrl:
```

```
RPC_URL });
    const resource = {
      url: "https://example.com/x402/moi/round-trip",
      description: "",
      mimeType: "",
    };

    // Seller: quote, then copy the resource URL in so the buyer can
sign over it.
    const server = new ServerScheme();
    const quoted = {
      scheme: "exact",
      network: MOI_DEVNET_CAIP2,
      asset: ASSET,
      amount: AMOUNT,
      payTo: PAY_TO!,
      maxTimeoutSeconds: 60,
      extra: {},
    };
    const [requirements] = await
server.enrichPaymentRequiredResponse({
      requirements: [quoted],
      resourceInfo: resource,
    } as never);

    // Buyer: transfer on chain, wait for the receipt, sign the claim.
    const client = new ClientScheme(signer);
    const { x402Version, payload } = await
client.createPaymentPayload(2, requirements);
    const paymentPayload = { x402Version, resource, accepted:
requirements, payload } as never;

    // Facilitator: read the transfer back and settle it, once.
    const facilitator = new
FacilitatorScheme(createMoiChainReader({ rpcUrl: RPC_URL }));
    const first = await facilitator.settle(paymentPayload,
requirements);
    expect(first.success, first.errorMessage).toBe(true);
    expect(first.payer).toBe(signer.address);
    expect(first.transaction).toBe((payload as { claim: { txHash:
string } }).claim.txHash);

    const second = await facilitator.settle(paymentPayload,
requirements);
    expect(second.success).toBe(false);
    expect(second.errorReason).toBe("moi_exact_already_spent");
  }, 120_000);
});
```

### 2.30 `typescript/.changeset/moi-exact-mechanism.md`

**Context.** Changesets entry (`minor`) in the directory Changesets actually reads; it was moved from the repo root.

```
---
"@x402/moi": minor
---

Add the MOI mechanism for the `exact` scheme. MOI uses the `upfront`
payment flow: the buyer
settles a protocol-native asset transfer first and proves it with a
signed claim bound to one
resource and one network. The facilitator holds no keys and confirms
the transfer by reading it
back off the chain; a JSON-RPC chain reader is included.
```

### 2.31 `e2e/config/mechanisms_moi.json`

**Context.** The e2e catalog entry: env vars per role, the devnet RPC default the harness injects as `MOI_RPC_URL`, and one route priced in atomic KMOI.

```
{
  "env": {
    "SERVER_MOI_ADDRESS": { "required": true, "roles": ["server"] },
    "CLIENT_MOI_MNEMONIC": { "required": true, "roles": ["client"] }
  },
  "testnet": {
    "name": "MOI Voyage Devnet",
    "caip2": "moi:devnet",
    "rpcUrlDefault": "https://dev.voyage-rpc.moi.technology/devnet/"
  },
  "routes": {
    "/exact/moi": {
      "scheme": "exact",
      "sdks": ["typescript"],
      "paymentFlow": "upfront",
      "price": { "asset":
"0x1080fffe4cd973c4eb83cdb8870c0de209736270491b7acc99873da100000000",
"amount": "1000" }
    }
  }
}
```

### 2.32 `.github/workflows/publish_npm_scoped_x402_moi.yml`

**Context.** Publish workflow copied from Stellar's OIDC pattern, so the package is released the same way as the others.

```
name: Publish @x402/moi package to NPM

on:
```

```
  workflow_dispatch:

jobs:
  publish-npm-x402-moi:
    if: github.repository == 'x402-foundation/x402' && github.ref ==
'refs/heads/main'
    runs-on: ubuntu-latest
    environment: npm
    permissions:
      contents: read
      id-token: write
    steps:
      - uses:
actions/checkout@de0fac2e4500dabe0009e67214ff5f5447ce83dd # v6.0.2

      - name: Setup pnpm
        uses: pnpm/action-
setup@0e279bb959325dab635dd2c09392533439d90093 # v6.0.8
        with:
          version: 11.1.1

      - uses: actions/setup-
node@48b55a011bda9f5d6aeb4c2d9c7362e8dae4041e # v6.4.0
        with:
          node-version: "24"
          registry-url: "https://registry.npmjs.org"

      - name: Update npm for OIDC trusted publishing
        run: npm install -g npm@11.14.1 --ignore-scripts

      - name: Configure npm for trusted publishing
        run: npm config delete always-auth 2>/dev/null || true

      - name: Install and build
        working-directory: ./typescript
        run: |
          pnpm install --frozen-lockfile --ignore-scripts
          pnpm -r --filter=@x402/core --filter=@x402/moi run build

      - name: Publish @x402/moi package
        working-directory: ./typescript/packages/mechanisms/moi
        run: |
          # Get package information directly
          PACKAGE_NAME=$(node -p "require('./package.json').name")
          PACKAGE_VERSION=$(node -p
"require('./package.json').version")

          echo "Package: $PACKAGE_NAME@$PACKAGE_VERSION"
```

```
echo "Publishing to NPM (main branch)"
pnpm publish --provenance --access public
```

## 3. Edits to existing e2e files

### e2e scheme registration

The e2e README requires four edits for a new network: the catalog file (`e2e/config/mechanisms_moi.json`, included) and one registration in each TypeScript role module. These three files exist upstream, so they are edits, patterned on the Stellar entries around them. The harness injects `MOI_RPC_URL` from the catalog's `rpcUrlDefault`, so no operator has to set it for devnet.

#### *e2e/servers/typescript/config.ts*

Import, alphabetically with the others:

```
import { ExactMoiScheme as ExactMoiServerScheme } from
"@x402/moi/exact/server";
```

Inside `registerFamilySchemes`, a new case:

```
    case "moi":
      server.register(pattern, new ExactMoiServerScheme());
      return;
```

#### *e2e/clients/typescript/client.ts*

Imports:

```
import { createMoiSigner, type ClientMoiSigner } from "@x402/moi";
import { ExactMoiScheme as ExactMoiClientScheme } from
"@x402/moi/exact/client";
```

Signer, next to the Stellar block:

```
let moiSigner: ClientMoiSigner | undefined;
if (process.env.CLIENT_MOI_MNEMONIC) {
  moiSigner = await createMoiSigner(process.env.CLIENT_MOI_MNEMONIC, {
    rpcUrl: process.env.MOI_RPC_URL,
  });
}
```

Registration, next to the Stellar block:

```
if (moiSigner) {
  schemes.push({
    network: networkCaip2Pattern("moi"),
    client: new ExactMoiClientScheme(moiSigner),
  });
}
```

### *e2e/facilitators/typescript/index.ts*

The MOI facilitator holds no keys, so unlike the other networks it is gated on the RPC URL rather than on a private key.

Imports:

```
import { createMoiChainReader } from "@x402/moi";
import { ExactMoiScheme as ExactMoiFacilitatorScheme } from
"@x402/moi/exact/facilitator";
```

Network and RPC constants, with the others:

```
const MOI_NETWORK = resolveNetworkCaip2("moi");
const MOI_RPC_URL = process.env.MOI_RPC_URL;
```

Registration:

```
if (MOI_RPC_URL) {
  facilitator.register(
    MOI_NETWORK as Network,
    new ExactMoiFacilitatorScheme(createMoiChainReader({ rpcUrl:
MOI_RPC_URL })),
  );
  console.info(`MOI facilitator reading ${MOI_NETWORK} at $
{MOI_RPC_URL}`);
}
```

Health check and startup banner, if the file lists networks there:

```
    moiNetwork: MOI_RPC_URL ? MOI_NETWORK : "(not configured)",
```

```
║  MOI Network: ${MOI_RPC_URL ? MOI_NETWORK : "(not configured)"} ║
```

The in-memory spent store is used when no `spentStore` option is passed; that is acceptable for the e2e harness and nowhere else.

## 4. Edits to existing example files

**`examples/typescript/*/advanced/all_networks.*`**

These three files already exist upstream, so they are edits rather than new files. Each keeps its networks in alphabetical order; `moi` goes after `hedera` and before `near`.

***clients***

```
import { createMoiSigner } from "@x402/moi";
import { ExactMoiScheme } from "@x402/moi/exact/client";

const moiMnemonic = process.env.CLIENT_MOI_MNEMONIC;
const moiRpcUrl = process.env.MOI_RPC_URL;

if (moiMnemonic) {
  const moiSigner = await createMoiSigner(moiMnemonic, { rpcUrl:
moiRpcUrl });
  client.register("moi:*", new ExactMoiScheme(moiSigner));
  console.log("Registered MOI client scheme");
}
```

`rpcUrl` defaults to Voyage devnet when unset. The signer's wallet is connected to that node, because the buyer half submits its own transfer through it.

***servers***

```
import { ExactMoiScheme } from "@x402/moi/exact/server";

const MOI_NETWORK = "moi:devnet" as const;
const moiAddress = process.env.SERVER_MOI_ADDRESS;

if (moiAddress) {
  server.register(MOI_NETWORK, new ExactMoiScheme());
}
```

The payment-requirements entry keys on `network: MOI_NETWORK, payTo: moiAddress`. Prices are `{ asset, amount }` in atomic units: a native asset carries no decimals or symbol on chain, so a money string cannot be resolved.

***facilitator***

```
import { createMoiChainReader, ExactMoiScheme } from
"@x402/moi/exact/facilitator";

const MOI_NETWORK = "moi:devnet";
const moiRpcUrl = process.env.MOI_RPC_URL;

facilitator.register(
  MOI_NETWORK,
  new ExactMoiScheme(createMoiChainReader({ rpcUrl: moiRpcUrl })),
);
```

The constructor takes a `MoiChainReader`, not a signer array — this facilitator only reads. `createMoiChainReader` supplies the three reads over a JSON-RPC node with js-moi-sdk: signature verification, deriving a participant identifier from a public key, and reading a settled transfer back via `moi.InteractionByHash` plus `moi.InteractionReceipt`, POLO-decoding the operation's calldata for the beneficiary and amount. Pass a durable `spentStore` in the options for anything beyond a demo.

# 5. Verification record

## PR 3 — TypeScript mechanism package

Built and verified. The package mirrors `@x402/stellar`'s layout and implements the scheme spec.

```
x402/typescript/packages/mechanisms/moi/
├── package.json  tsconfig.json  tsup.config.ts
├── vitest.config.ts  vitest.integration.config.ts
├── eslint.config.js  .prettierrc  .prettierignore
├── README.md  CHANGELOG.md
├── src/
│   ├── constants.ts  types.ts  utils.ts  shared.ts
│   ├── signer.ts  reader.ts  defaultAssets.ts  index.ts
│   └── exact/  index.ts + client/ server/ facilitator/
└── test/  unit/ (facilitator, reader)  integrations/
x402/e2e/config/mechanisms_moi.json
x402/.github/workflows/publish_npm_scoped_x402_moi.yml
```

***Verified 10 September 2026***

Against the real dependencies installed from npm — `@x402/core@2.25.0`, `js-moi-sdk@0.9.0-rc2`, `js-polo@0.1.5` — with upstream's `tsconfig.base.json`:

- `tsc` clean. `eslint` clean under the config copied verbatim from `@x402/stellar`. Prettier clean.
- 36 unit tests pass (facilitator, reader, server). Coverage thresholds (70%) met.
- The live read-back integration tests pass against Voyage devnet: a KMOI transfer settled at sequence `0x4f` of `0x…bfa45bc0` comes back through `moi.InteractionByHash` and `moi.InteractionReceipt` exactly as spec rule 7 describes, the node's POLO calldata decodes to beneficiary `0x…ea3f6fdc` and amount `250`, and an unknown hash reads as `null`.
- The round trip (pay, prove, settle once, second settle refused) is written and skips without `CLIENT_MOI_MNEMONIC` and `SERVER_MOI_ADDRESS`. It has not yet been run with a funded wallet.

***Second pass, 10 September***

- **A chain reader ships.** `src/reader.ts` — `createMoiChainReader({ rpcUrl | provider })`, `JsonRpcMoiChainReader`, `decodeTransferCalldata`. The package previously defined the `MoiChainReader` interface and left every deployer to implement it. Signature verification goes through js-moi-sdk's `Signer.verify` (blake2b-256 digest, ECDSA secp256k1) via a verify-only subclass; identifier derivation mirrors `Wallet.getIdentifier` (v0 tag, compressed key bytes 1–25 as the fingerprint, variant 0); the read-back does both RPC calls and POLO-decodes the `Transfer` calldata. A missing interaction reads as `null`; any other failure propagates

and the facilitator turns it into `moi_exact_chain_read_failed` rather than throwing.

- **`keyId` is enforced**, with a test. STATUS previously claimed it was pinned; the code only documented it as informational.
- **The client waits for the receipt** (`interaction.wait()`) and checks the interaction-level status, as spec step 4 says, instead of inspecting `result()` for an error field.
- **`createMoiSigner` connects the wallet to a node.** It previously did not, so the client half could not actually submit a transfer. Signature is now `(mnemonic, { rpcUrl | provider, derivationPath })`; `keyId` is fixed at 0.
- **The reader is told the quoted asset** (`readTransfer(txHash, asset)`) so it can pick the right operation from a multi-operation interaction.
- Unused interface parameters dropped; type imports reordered; `eslint.config.js` replaced with upstream's so the package is linted the way CI will lint it.
- `e2e/config/mechanisms_moi.json`: KMOI's post-0.13.0 asset id, and `CLIENT_MOI_MNEMONIC` in place of a private key (the SDK's wallet path is mnemonic-first).

***Claims checked against source***

go-moi `develop` at `01b0ae58` (2026-09-07, "feat: ix gas payer"), js-moi-sdk `0.9.0-rc2` from npm, and live Voyage devnet (`net.Version` = `0.12.0`):

| Claim in spec / code | Where it holds |
|---|---|
| `ASSET_INVOKE` is op type 5 | go-moi `IxOpType` (`IxAssetAction` is the 6th constant, iota 5); SDK `OpType.ASSET_INVOKE = 5` |
| receipt `status` 0 is success | go-moi `ReceiptOk = iota` (0), then `ReceiptStateReverted`, `ReceiptInsufficientFuel` |
| receipt carries no beneficiary/amount | go-moi `Receipt { IxHash, Status, FuelUsed, IxOps }`; RPC adds `from`, `ts_hash`, `participants` |
| callsite is `"Transfer"` | go-moi `TransferEndpoint = "Transfer"`; MAS0 Coco endpoint `Transfer(beneficiary Identifier, amount U256)`; SDK `MAS0.Endpoint.TRANSFER` |
| calldata is a POLO document `{beneficiary, amount}` | go-moi `TransferParams` with `polo:"beneficiary"`, `polo:"amount"`, encoded via `PolorizeDocument`; decoded live |

| Claim in spec / code | Where it holds |
|---|---|
| KMOI is MASN; new id `0x1080fffe4cd9…` | go-moi `MASN = 0xFFFE`, `KMOITokenAssetID = CreateAssetIDFromString("KMOI", 0, MASN, Systemic)`; SDK `KMOI_ASSET_ID` |
| MASN transfers use the same callsite/calldata | go-moi `kmoiReservedEndpoints` reserves only Mint/Burn/metadata; SDK `MASNAssetLogic.transfer` mirrors `MAS0AssetLogic.transfer` |
| minimum fuel price 50 anu, SDK default clears it | go-moi `DefaultIxPriceLimit = 50`; SDK `DEFAULT_FUEL_PRICE = 50` |
| `sender { id, sequence_id, key_id }`, `fee_payer`, `notary` on participants | go-moi `RPCSender`, `RPCInteraction.FeePayer json:"fee_payer"`, `IxParticipant.Notary` |
| envelope is POLO-encoded and signed whole | go-moi `PayloadForSignature() = polo.Polorize(ix.inner)` |
| identifier = tag, flags, 24-byte key slice, variant | go-moi participant v0 layout `[tag:1][flags:1][standard:2][fingerprint:24][variant:4]`; SDK `getIdentifier` uses `pubkey[1..25)` |
| signatures are ECDSA secp256k1 over the claim bytes | SDK `ECDSA_S256.sign/verify`: blake2b-256 of the message, noble secp256k1 |
| a refused transfer still returns a hash and receipt | go-moi `ReceiptStateReverted`; RPC `InteractionReceipt` returns for reverted interactions |

Two corrections made to the spec from this pass: the identifier layout has two standard bytes between the flags and the fingerprint (the spec listed 30 of the 32 bytes), and the RPC response spells the participant list `ix_participants` (the signing schema says `participants`).

***Third pass, 10 September (Stellar comparison)***

- Facilitator `getExtra()` now returns `{ areFeesSponsored: false }`, the key Stellar puts on the wire; the spec's `PaymentRequirements` example documents it.
- KMOI is a built-in default asset (`KMOI_ASSET_ID`, 9 decimals, symbol `KMOI`), so core's display path stops assuming 6 decimals, `"0.10 KMOI"` prices convert, and the client exposes `findDefaultAsset` for spend controls. Dollar prices are refused unless the developer calls `registerMoneyParser`, the same API Stellar offers.

- `test/unit/server.test.ts` added (8 tests). Suite 36/36 against js-moi-sdk 0.9.0-rc2 and @x402/core 2.25.0; ESLint (Stellar's plugin set) and Prettier clean.
- Client `send()` passes `fuel_limit/fuel_price` only when set; rc2 defaults the price to 50 anu, the node's new minimum. Unused `MOI_MAINNET_CAIP2` and a dead type removed; `MAS0_TRANSFER_CALLSITE` renamed `TRANSFER_CALLSITE`.
- `js-polo` pinned `^0.1.5`; publish workflow rewritten to Stellar's OIDC pattern; changeset moved to `typescript/.changeset/`; README gains the in-process facilitator adapter and the corrected pricing paragraph.

***e2e and housekeeping***

The e2e README requires four edits for a new network, not one: the catalog file plus a registration in `e2e/servers/typescript/config.ts`, `e2e/clients/typescript/client.ts` and `e2e/facilitators/typescript/index.ts`. The earlier note that the catalog alone suffices was wrong. The three edits are written out in `E2E-REGISTRATION.md`. The changeset lives at `typescript/.changeset/moi-exact-mechanism.md` (moved from the repo root, which Changesets does not read).

***Before filing: run against moipod v0.13.0***

Live at 00:00 UTC on 11 September. The round trip has not run on a live network.

1. `MOI_ASSET_ID` defaults to the SDK's `KMOI_ASSET_ID`, which is already the post-0.13.0 id. Against a pre-0.13.0 node, pass the old id (`0x108000004cd973c4eb83cdb8870c0de209736270491b7acc99873da100000000`).
2. Fuel price is settled: the SDK's default 50 anu equals the new minimum. The client passes `fuel_limit/fuel_price` to `InteractionContext.send()` only when the caller sets them, so unset values fall through to the SDK's own defaults.
3. Run `pnpm test:integration` with `CLIENT_MOI_MNEMONIC` (funded) and `SERVER_MOI_ADDRESS`.
4. If devnet is reset by the upgrade, refresh the `SETTLED` fixture in the integration test with any settled `Transfer` and its decoded fields (the unit fixture in `reader.test.ts` is self-contained and keeps working).
5. Note for PR 2, not this package: js-moi-sdk 0.9.0-rc2 still sends the fee payer as `payer` while go-moi `develop` reads `fee_payer`. This scheme sets no fee payer, so nothing here depends on it, but the spec's anatomy line should be re-checked on the wire once 0.13.0 is live.

***What the rebuild fixed***

The earlier draft in `../../moi-x402/` diverged from the spec in nine ways. All are corrected here.

- The resource binding was dead. The client read `requirements.extra.resource`, which nothing populated, so every claim signed `resource: ""` and the facilitator never checked it. The server half now copies the URL in through `enrichPaymentRequiredResponse`, the only hook that receives the `ResourceInfo`; the client refuses to build a payload without it; the facilitator compares against `paymentPayload.resource.url` and fails closed when it is absent.
- The validity window was always wider than quoted: `validAfter = now - 5` with `validBefore = now + maxTimeoutSeconds`. The client now clamps the window and the facilitator rejects any window exceeding the quote.
- `getExtra()` emitted `flow`. The reserved key is `paymentFlow`, and core writes it onto the wire itself, so the facilitator no longer sets it at all.
- The callsite was matched with `/transfer/i`, which would accept `TransferFrom`. It is now compared exactly against `Transfer`.
- Amounts passed through `Number()`, losing precision above 2^53. All quantities are bigint, and `toBigInt` accepts both the hex the RPC returns and the decimal strings payloads carry.
- Replay used a separate has-then-add. Under this flow two concurrent settlements can both read the same settled transfer before either records it, so the store now exposes one atomic `reserve`.
- All three classes were named `MoiExactScheme`. They follow the upstream convention as `ExactMoiScheme`, one per role module.
- `js-moi-sdk` was pinned at `^0.7.1`; the package depends on `^0.9.0-rc2`.
- There were no tests.

Left as it was, having checked it: `defaultAssetTransferMethod = "default"` is correct. The interface documents that value for a scheme with no on-wire choice of transfer method, and core strips the key from the wire when it is used. `areFeesSponsored` also stays — it is not a `SchemeNetworkFacilitator` member, but `@x402/stellar` declares it as a class property, so it is the house convention rather than an invention.

***Sequencing***

This PR goes after the scheme spec merges, which itself waits on the CASA namespace. The `examples/` entries and a changeset are the remaining pieces, both trivial once the package name is fixed upstream.